

Analisi e progetto di algoritmi

Esercizi di programmazione dinamica

Notazione

Le sequenze di numeri interi vengono indicate con le lettere maiuscole, ad esempio $S = \langle 1, 7, 4, 9 \rangle$. L' i -esimo prefisso di una sequenza $X = \langle x_1, x_2, \dots, x_n \rangle$ viene indicato con $X_i = \langle x_1, x_2, \dots, x_i \rangle$, ad esempio considerata la sequenza S definita precedentemente, il prefisso di indice 2 è $S_2 = \langle 1, 7 \rangle$. Secondo questa definizione risulta evidente che $X_0 = \epsilon$, ossia la sequenza vuota. È definita sottosequenza una qualsiasi successione di elementi (non necessariamente contigui) appartenenti alla medesima sequenza. Ad esempio $\langle 7, 4 \rangle$ e $\langle 7, 9 \rangle$ sono sottosequenze di S , mentre $\langle 7, 9, 4 \rangle$ non è una sottosequenza di S .

Ogni esercizio presenta la seguente struttura:

1. definizione del problema principale PB e di una sua versione ridotta PBR
2. definizione dei sottoproblemi comuni e delle variabili descrittive
3. *possibile introduzione di un problema ausiliario e relativi sottoproblemi*
4. calcolo delle soluzioni dei sottoproblemi tramite equazioni di ricorrenza (passo ricorsivo e caso base)
5. definizione della soluzione del problema (eventualmente in funzione delle soluzioni dei sottoproblemi del problema ausiliario)
6. presentazione dello pseudocodice¹ e della complessità dell'algoritmo

¹Si assume che alcune subroutine utilizzate siano date, come per esempio la subroutine MAX che calcola il massimo fra due o più valori.

1 Più lunga sottosequenza comune (LCS)

1.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini UNA tra le più lunghe sottosequenze comuni a X e Y

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, una tra le più lunghe sottosequenze comuni è la sottosequenza $\langle 2, 4, 21, 14, 1 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X e Y

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, la lunghezza di una tra le più lunghe sottosequenze comuni è 5.

1.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la coppia (X, Y) ma una coppia di prefissi di tali sequenze.

Il **sottoproblema di dimensione** (i, j) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni al prefisso X_i e al prefisso Y_j

Dato che $0 \leq i \leq m$ e $0 \leq j \leq n$, si ottengono $(m+1) \cdot (n+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j) , la variabile ad esso associata è $c_{i,j}$ ed è così definita:

$c_{i,j} = \text{lunghezza di una tra le più lunghe sottosequenze comuni a } X_i \text{ e } Y_j$

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(2, 2)$, il valore della variabile $c_{2,2}$ si otterrà utilizzando le variabili $c_{0,0}$, $c_{0,1}$, $c_{1,0}$, $c_{1,1}$, $c_{1,2}$ e $c_{2,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) ?

1.3 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

1.3.1 Caso base: (i, j) con $i = 0 \vee j = 0$

Il caso base si ha per un qualunque sottoproblema di dimensione (i, j) con $i = 0 \vee j = 0$, ossia quando uno dei due prefissi considerati è la sequenza vuota. In questo caso, è facile ottenere il valore della variabile $c_{i,j}$ in quanto la lunghezza di una tra le più lunghe sottosequenze comuni fra una qualsiasi sequenza e la sequenza vuota è 0 (ossia la lunghezza della sequenza vuota). Per questa ragione, il caso base è scrivibile come:

$$c_{i,j} = 0 \quad \text{se } i = 0 \vee j = 0$$

1.3.2 Passo ricorsivo: (i, j) con $i > 0$ e $j > 0$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, j) con $i > 0 \wedge j > 0$, ossia quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota. I dati disponibili per calcolare $c_{i,j}$ sono: l'input X ed in particolare l'elemento x_i , l'input Y ed in particolare l'elemento y_j e tutte le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$. Per calcolare $c_{i,j}$ e quindi risolvere il problema di dimensione (i, j) è necessario applicare il *teorema della proprietà della sottostruttura ottima della LCS*, distinguendo i seguenti due casi:

- $x_i = y_j$: se i due elementi considerati sono identici allora la lunghezza della più lunga sottosequenza comune fra X_i e Y_j è uguale alla lunghezza della più lunga sottosequenza comune fra X_{i-1} e Y_{j-1} (ossia il valore di $c_{i-1,j-1}$) aumentata di uno (l'elemento comune x_i è accodato ad una più lunga sottosequenza comune a X_{i-1} e Y_{j-1} correlata al sottoproblema di dimensione $(i-1, j-1)$ e che ha quindi la lunghezza $c_{i-1,j-1}$). In altri termini, se Z_k è una LCS fra X_i e Y_j , allora $z_k = x_i = y_j$ e Z_{k-1} è una LCS fra X_{i-1} e Y_{j-1} . Il tutto è esemplificato dalla Figura 1 e può essere scritto come:

$$c_{i,j} = 1 + c_{i-1,j-1} \quad \text{se } x_i = y_j$$

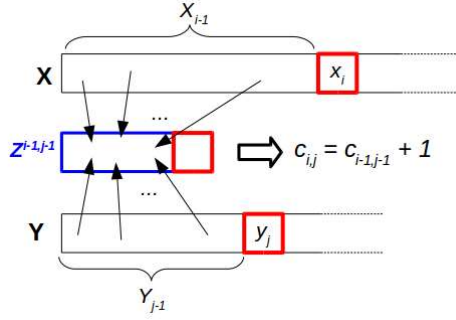


Figura 1: Sottosequenza comune più lunga: passo ricorsivo nel caso in cui $x_i = y_j$. L'elemento x_i viene accodato a $Z^{i-1,j-1}$ che è una più lunga sottosequenza comune a X_{i-1} e Y_{j-1}

- $x_i \neq y_j$: se i due elementi considerati sono differenti, allora la lunghezza della più lunga sottosequenza comune è data dalla soluzione di uno dei sottoproblemi di dimensione minore. Siccome gli elementi finali dei due prefissi considerati sono differenti, non possono appartenere entrambi alla soluzione Z_k e risulta necessario considerare i seguenti due casi, sulla base di come è fatto l'ultimo elemento z_k della soluzione Z_k :
 - se $z_k \neq x_i$, allora si deve considerare la soluzione $c_{i-1,j}$, ossia la soluzione del sottoproblema di dimensione $(i-1, j)$ con input il prefisso X_{i-1} e il prefisso Y_j (si è tolto da X_i l'elemento x_i e si è mantenuto l'elemento y_j)
 - se $z_k \neq y_j$, allora si deve considerare la soluzione $c_{i,j-1}$, ossia la soluzione del sottoproblema di dimensione $(i, j-1)$ con input il prefisso X_i e il prefisso Y_{j-1} (si è mantenuto l'elemento x_i e si è tolto da Y_j l'elemento y_j)

Siccome z_k non è noto, non possiamo sapere a priori quale dei due casi si verifichi. Quindi consideriamo la soluzione di entrambi i sottoproblemi di dimensione $(i-1, j)$ e $(i, j-1)$ e scegliamo quella di valore massimo. Formalmente, possiamo scrivere:

$$c_{i,j} = \max\{c_{i-1,j}, c_{i,j-1}\} \quad \text{se } x_i \neq y_j$$

Il passo ricorsivo è quindi scrivibile come:

$$c_{i,j} = \begin{cases} 1 + c_{i-1,j-1} & \text{se } x_i = y_j \\ \max\{c_{i,j-1}, c_{i-1,j}\} & \text{se } x_i \neq y_j \end{cases}$$

1.4 Soluzione del problema

Una volta calcolati i valori $c_{0,0}, c_{0,1}, \dots, c_{m,n}$ si hanno a disposizione tutte le lunghezze delle sottosequenze comuni massimali fra qualsiasi prefisso di X e qualsiasi prefisso di Y . La soluzione al problema PBR è:

$$c_{m,n}$$

		0	1	2	3	4	5	6	7	8	j
		ε	2	7	4	23	21	14	1	8	y_j
0	ε	0	0	0	0	0	0	0	0	0	
1	2	0	1	1	1	1	1	1	1	1	
2	4	0	1	1	2	2	2	2	2	2	
3	7	0	1	2	2	2	2	2	2	2	
4	11	0	1	2	2	2	2	2	2	2	
5	21	0	1	2	2	2	3	3	3	3	
6	14	0	1	2	2	2	3	4	4	4	
7	1	0	1	2	2	2	3	4	5	5	
i	x_i										

Figura 2: Sottosequenza comune più lunga: matrice di tutti i $c_{i,j}$ ottenuti considerando le sequenze $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

Esempio Date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, tutti i valori $c_{i,j}$ relativi sono contenuti nella matrice presentata in Figura 2.

1.5 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 1.3, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j}$ di ogni sottoproblema. L'algoritmo 1 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LCSRIC}(m, n)$ (si suppone di aver accesso alle sequenze X e Y). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

1.6 Implementazione bottom-up

L'algoritmo 2 presenta lo pseudocodice che calcola tutti i valori $c_{0,0}, c_{0,1}, \dots, c_{m,n}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 1.3. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m \cdot n)$ occupando $\Theta(m \cdot n)$ spazio in memoria (ossia una matrice che contiene i vari $c_{i,j}$).

Algorithm 1 Lunghezza di una più lunga sottosequenza comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: ricorsione).

```
procedure LCSRIC( $i, j$ )  
  if  $i = 0 \vee j = 0$  then  
    return 0  
  else  
    if  $x_i = y_j$  then  
      return  $1 + \text{LCSRIC}(i - 1, j - 1)$   
    else  
       $S_1 \leftarrow \text{LCSRIC}(i, j - 1)$   
       $S_2 \leftarrow \text{LCSRIC}(i - 1, j)$   
      return  $\text{MAX}(S_1, S_2)$ 
```

Algorithm 2 Lunghezza di una più lunga sottosequenza comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: bottom-up).

```
procedure LCS( $X, Y$ )  
  for  $i \leftarrow 0$  to  $m$  do  
     $c[i, 0] \leftarrow 0$   
  for  $j \leftarrow 1$  to  $n$  do  
     $c[0, j] \leftarrow 0$   
  for  $i \leftarrow 1$  to  $m$  do  
    for  $j \leftarrow 1$  to  $n$  do  
      if  $x_i = y_j$  then  
         $c[i, j] \leftarrow 1 + c[i - 1, j - 1]$   
      else  
         $c[i, j] \leftarrow \text{MAX}(c[i, j - 1], c[i - 1, j])$   
  return  $c[m, n]$ 
```

2 Più lunga sottosequenza comune a tre sequenze (LCS3)

2.1 Definizione del problema

Il problema PB è definito come segue:

date tre sequenze X, Y e W , rispettivamente di n, m e l numeri interi, si determini UNA tra le più lunghe sottosequenze comuni a X, Y e W

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$, $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$ e $W = \langle 4, 7, 11, 21, 1, 8 \rangle$, una tra le più lunghe sottosequenze comuni è la sottosequenza $\langle 4, 21, 1 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

date tre sequenze X, Y e W , rispettivamente di n, m e l numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X, Y e W

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$, $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$ e $W = \langle 4, 7, 11, 21, 1, 8 \rangle$, la lunghezza di una tra le più lunghe sottosequenze comuni è 3.

2.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la tripla (X, Y, W) ma una tripla di prefissi di tali sequenze.

Il **sottoproblema di dimensione** (i, j, h) è definito come segue:

date tre sequenze X, Y e W , rispettivamente di n, m e l numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni ai prefissi X_i, Y_j e W_h

Dato che $0 \leq i \leq n$, $0 \leq j \leq m$ e $0 \leq h \leq l$, si ottengono $(n+1) \cdot (m+1) \cdot (l+1)$ sottoproblemi (i, j e h possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j, h) , la variabile ad esso associata è $c_{i,j,h}$ ed è così definita:

$c_{i,j,h}$ = *lunghezza di una tra le più lunghe sottosequenze comuni a X_i, Y_j e W_h*

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j, h) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(1, 1, 1)$, il valore della variabile $c_{1,1,1}$ si otterrà utilizzando le variabili $c_{0,0,0}$, $c_{0,0,1}$, $c_{0,1,0}$, $c_{0,1,1}$, $c_{1,0,0}$, $c_{1,0,1}$ e $c_{1,1,0}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne

il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j, h) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_{0,0,0}, \dots, c_{i-1,j,h}, c_{i,j-1,h}, c_{i,j,h-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j,h}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j, h) ed, eventualmente, accodando gli elementi x_i, y_j e w_h estratti dall'input (X_i, Y_j, W_h) ?

2.3 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

2.3.1 Caso base: (i, j, h) con $i = 0 \vee j = 0 \vee h = 0$

Il caso base si ha per un qualunque sottoproblema di dimensione (i, j, h) con $i = 0 \vee j = 0 \vee h = 0$, ossia quando uno dei prefissi considerati è la sequenza vuota. In questo caso, è facile ottenere il valore della variabile $c_{i,j,h}$ in quanto la lunghezza di una tra le più lunghe sottosequenze comuni fra una qualsiasi sequenza e la sequenza vuota è 0 (ossia la lunghezza della sequenza vuota). Per questa ragione, il caso base è scrivibile come:

$$c_{i,j,h} = 0 \quad \text{se } i = 0 \vee j = 0 \vee h = 0$$

Si noti che, in totale, si hanno $(m+1) \cdot (l+1) + n \cdot (l+1) + n \cdot m$ sottoproblemi che possono essere ricondotti al caso base.

2.3.2 Passo ricorsivo: (i, j, h) con $i > 0 \wedge j > 0 \wedge h > 0$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, j, h) con $i > 0 \wedge j > 0 \wedge h > 0$, ossia quando si vanno a considerare tre prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$, $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ e $W_h = \langle w_0, w_1, \dots, w_h \rangle$ diversi dalla sequenza vuota. I dati disponibili per calcolare $c_{i,j,h}$ sono: l'input X ed in particolare l'elemento x_i , l'input Y ed in particolare l'elemento y_j , l'input W ed in particolare l'elemento w_h e tutte le variabili $\{c_{0,0,0}, \dots, c_{i-1,j,h}, c_{i,j-1,h}, c_{i,j,h-1}\}$. Per calcolare $c_{i,j,h}$ e quindi risolvere il problema di dimensione (i, j, h) è necessario applicare il *teorema della sottostruttura ottima*. È possibile adattare il teorema

utilizzato per il problema della più lunga sottosequenza comune a due sequenze (Sezione 1), andando quindi a distinguere i seguenti due casi:

- $x_i = y_j = w_h$: se i tre elementi considerati sono identici allora la lunghezza della più lunga sottosequenza comune fra X_i , Y_j e W_h è uguale alla lunghezza della più lunga sottosequenza comune fra X_{i-1} , Y_{j-1} e W_{h-1} (ossia il valore di $c_{i-1,j-1,h-1}$) aumentata di uno (l'elemento comune x_i è accaduto ad una più lunga sottosequenza comune a X_{i-1} , Y_{j-1} e W_{h-1} correlata al sottoproblema di dimensione $(i-1, j-1, h-1)$ e che ha quindi ha lunghezza $c_{i-1,j-1,h-1}$). Il tutto può essere scritto come:

$$c_{i,j,h} = 1 + c_{i-1,j-1,h-1} \quad \text{se } x_i = y_j = w_h$$

- $x_i \neq y_j \vee x_i \neq w_h \vee y_j \neq w_h$: se gli elementi considerati sono differenti, allora la lunghezza della più lunga sottosequenza comune è data dalla soluzione di uno dei sottoproblemi di dimensione minore. Siccome gli elementi finali dei prefissi considerati sono differenti, non possono appartenere tutti alla soluzione Z_k ed è necessario considerare i seguenti sottocasi, sulla base di come è fatto l'ultimo elemento z_k della soluzione Z_k :

- se $z_k \neq x_i$, allora si deve considerare la soluzione $c_{i-1,j,h}$, ossia la soluzione del sottoproblema di dimensione $(i-1, j, h)$ con input i prefissi X_{i-1} , Y_j e W_h (si è tolto da X_i l'elemento x_i e si sono mantenuti gli elementi y_j e w_h)
- se $z_k \neq y_j$, allora si deve considerare la soluzione $c_{i,j-1,h}$, ossia la soluzione del sottoproblema di dimensione $(i, j-1, h)$ con input i prefissi X_i , Y_{j-1} e W_h (si è tolto da Y_j l'elemento y_j e si sono mantenuti gli elementi x_i e w_h)
- se $z_k \neq w_h$, allora si deve considerare la soluzione $c_{i,j,h-1}$, ossia la soluzione del sottoproblema di dimensione $(i, j, h-1)$ con input i prefissi X_i , Y_j e W_{h-1} (si è tolto da W_h l'elemento w_h e si sono mantenuti gli elementi x_i e y_j)

Siccome z_k non è noto, non possiamo sapere a priori quale dei tre casi si verifichi. Quindi consideriamo la soluzione dei tre sottoproblemi di dimensione $(i-1, j, h)$, $(i, j-1, h)$ e $(i, j, h-1)$ e scegliamo quella di valore massimo. Formalmente, possiamo scrivere:

$$c_{i,j,h} = \max\{c_{i-1,j,h}, c_{i,j-1,h}, c_{i,j,h-1}\} \quad \text{se } x_i \neq y_j \vee x_i \neq w_h \vee y_j \neq w_h$$

Il passo ricorsivo è quindi scrivibile come:

$$c_{i,j,h} = \begin{cases} 1 + c_{i-1,j-1,h-1} & \text{se } x_i = y_j = w_h \\ \max\{c_{i-1,j,h}, c_{i,j-1,h}, c_{i,j,h-1}\} & \text{altrimenti} \end{cases}$$

2.4 Soluzione del problema

Una volta calcolati i valori $c_{0,0,0}, c_{0,0,1}, \dots, c_{n,m,l}$ si hanno a disposizione tutte le lunghezze delle sottosequenze comuni massimali fra qualsiasi prefisso di X , qualsiasi prefisso di Y e qualsiasi prefisso di W . La soluzione al problema PBR è:

$$c_{n,m,l}$$

2.5 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 2.3, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j,h}$ di ogni sottoproblema. L'algoritmo 3 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LCS3RIC}(n, m, l)$ (si suppone di aver accesso alle sequenze X , Y e W). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 3 Lunghezza di una più lunga sottosequenza comune fra tre sequenze X , Y e W di lunghezza n , m e l rispettivamente (tecnica: ricorsione).

```
procedure LCS3RIC( $i, j, h$ )
  if  $i = 0 \vee j = 0 \vee h = 0$  then
    return 0
  else
    if  $x_i = y_j = w_h$  then
      return  $1 + \text{LCS3RIC}(i - 1, j - 1, h - 1)$ 
    else
      return  $\text{MAX}(\text{LCS3RIC}(i - 1, j, h), \text{LCS3RIC}(i, j - 1, h), \text{LCS3RIC}(i, j, h - 1))$ 
```

2.6 Implementazione bottom-up

L'algoritmo 4 presenta lo pseudocodice che calcola tutti i valori $c_{0,0,0}, c_{0,0,1}, \dots, c_{n,m,l}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 2.3. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(n \cdot m \cdot l)$ occupando, però, $\Theta(n \cdot m \cdot l)$ spazio in memoria (ossia una matrice che contiene i vari $c_{i,j,h}$).

Algorithm 4 Lunghezza di una più lunga sottosequenza comune fra tre sequenze X , Y e W di lunghezza n , m e l rispettivamente (tecnica: bottom-up).

```
procedure LCS3( $X, Y, W$ )
  for  $j \leftarrow 0$  to  $m$  do
    for  $h \leftarrow 0$  to  $l$  do
       $c[0, j, h] \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
    for  $h \leftarrow 0$  to  $l$  do
       $c[i, 0, h] \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
    for  $j \leftarrow 1$  to  $m$  do
       $c[i, j, 0] \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
    for  $j \leftarrow 1$  to  $m$  do
      for  $h \leftarrow 1$  to  $l$  do
        if  $x_i = y_j = w_h$  then
           $c[i, j, h] \leftarrow 1 + c[i - 1, j - 1, h - 1]$ 
        else
           $c[i, j, h] \leftarrow \text{MAX}(c[i - 1, j, h], c[i, j - 1, h], c[i, j, h - 1])$ 
  return  $c[n, m, l]$ 
```

3 Più lunga sottosequenza crescente (LGS)

3.1 Definizione del problema

Il problema PB è definito come segue:

data una sequenza X di m numeri interi, si determini UNA tra le più lunghe sottosequenze crescenti di X

Ad esempio, data $X = \langle 2, 4, 7, 6, 11, 13, 21, 14, 1 \rangle$, una tra le più lunghe sottosequenze crescenti di X è la sottosequenza $\langle 2, 4, 6, 11, 13, 21 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti di X

Ad esempio, data $X = \langle 2, 4, 7, 6, 11, 13, 21, 14, 1 \rangle$, la lunghezza di una tra le più lunghe sottosequenze crescenti di X è 6.

3.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la sequenza X ma un suo prefisso.

Il **sottoproblema di dimensione** (i) è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti di X_i

Dato che $0 \leq i \leq m$, si ottengono $(m + 1)$ sottoproblemi (i può valere 0 in quanto si deve considerare anche il caso in cui il prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i) , la variabile ad esso associata è c_i ed è così definita:

$$c_i = \text{lunghezza di una tra le più lunghe sottosequenze crescenti di } X_i$$

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione (3) , il valore della variabile c_3 si otterrà utilizzando le variabili c_0 , c_1 e c_2 . Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_0, \dots, c_{i-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione c_i andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i) ed, eventualmente, accodando l'elemento x_i estratto dall'input X_i ?

Purtroppo, però, il problema così definito non è risolvibile: **ci manca informazione**. Date solamente le variabili $\{c_0, \dots, c_{i-1}\}$ e il prefisso X_i (del quale, in realtà, ci interessa solo l'elemento x_i), non c'è alcun modo per poter comprendere se l'elemento x_i possa essere accodato alle sottosequenze crescenti relative ai sottoproblemi di dimensione minore a i : non sappiamo con quale elemento termini ognuna di queste sottosequenze ma ne conosciamo solo la lunghezza.

Risulta necessario introdurre un problema ausiliario, nel quale introdurre l'informazione mancante necessaria.

3.3 Problema ausiliario P' di PBR

Il problema ausiliario P' è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti di X e che termina con x_m

In altre parole, si richiede che la sottosequenza crescente più lunga termini con l'ultimo elemento della sequenza in input.

Come il problema ridotto PBR, anche il problema ausiliario contiene $m+1$ diversi sottoproblemi, ognuno associato ad una differente variabile. Il sottoproblema di dimensione (i) è definito come:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti di X_i e che termina con x_i

ed è associato alla variabile c_i così definita:

$c_i =$ *lunghezza di una tra le più lunghe sottosequenze crescenti di X_i e che termina con x_i*

Si noti quindi che per un qualunque sottoproblema di dimensione (s), solo imponendo che la soluzione c_s si riferisca ad una sottosequenza crescente di X_s che termina con x_s è possibile stabilire se un altro elemento x_u con $u > s$ possa essere accodato a tale sottosequenza (andando a verificare che $x_s < x_u$).

3.4 Equazioni di ricorrenza

É ora possibile risolvere il problema ausiliario P' andando a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

3.4.1 Caso base: (i) con $i \leq 1$

Il caso base si ha per un qualunque sottoproblema di dimensione (i) con $i = 0 \vee i = 1$, ossia quando il prefisso considerato è la sequenza vuota oppure è una sequenza composta da un singolo elemento. In entrambi i casi, è facile ottenere il valore della variabile c_i : la lunghezza di una tra le più lunghe sottosequenze crescenti della sequenza vuota è 0 mentre la lunghezza di una tra le più lunghe sottosequenze crescenti di una sequenza composta da un singolo elemento e che termina con quell'elemento è 1. Il caso base $i = 0$, comunque, risulta non necessario in quanto è possibile utilizzare, come caso base, solamente il caso $i = 1$. Per questa ragione, il caso base è scrivibile come:

$$c_i = 1 \quad \text{se } i = 1$$

3.4.2 Passo ricorsivo: (i) con $i > 1$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i) con $i > 1$, ossia quando si considera un prefisso della sequenza X in input di almeno due elementi. I dati disponibili per calcolare c_i sono: l'input X ed in particolare l'elemento x_i e tutte le variabili $\{c_0, \dots, c_{i-1}\}$. Si ricorda che i valori $c_1, c_2, \dots, c_{i-1}$ rappresentano le lunghezze delle più lunghe sottosequenze crescenti di $X_1, X_2, \dots, X_{i-1}$ che terminano, rispettivamente, con $x_1, x_2, \dots, x_{i-1}$. Tra queste ci saranno alcune sottosequenze alle quali possiamo accodare x_i (in quanto maggiore dell'ultimo elemento) e altre alle quali l'elemento x_i non può essere accodato (in quanto minore dell'ultimo elemento). Se prendiamo la più lunga sottosequenza crescente alla quale possiamo attaccare x_i e accodiamo ad essa x_i , otteniamo la più lunga sottosequenza crescente di X_i che termina con x_i . La lunghezza di tale sottosequenza sarà uguale alla lunghezza della sottosequenza crescente alla quale abbiamo accodato x_i aumentata di 1. Il passo ricorsivo è quindi scrivibile come:

$$c_i = 1 + \max\{c_h \mid 1 \leq h < i \wedge x_h < x_i\}$$

Poiché può accadere che l'insieme $\{c_h \mid 1 \leq h < i \wedge x_h < x_i\}$ sia vuoto (il che corrisponde al fatto che l'elemento x_i è minore di tutti gli elementi precedenti e, quindi, non può essere accodato a nessuna più lunga sottosequenza crescente relativa a sottoproblemi di dimensione minore), assumiamo per definizione che $\max\{\emptyset\} = 0$, così che il corrispondente valore di c_i risulti uguale a 1.

3.5 Soluzione del problema

Una volta calcolati i valori $c_0, c_1, \dots, c_m$ si hanno a disposizione tutte le lunghezze di una tra le più lunghe sottosequenze crescenti dei vari prefissi di X e che terminano con l'ultimo elemento del prefisso. La soluzione del problema ausiliario P' è c_m mentre quella del problema ridotto PBR è

$$\max\{c_i \mid 1 \leq i \leq m\}$$

Esempio Data $X = \langle 2, 4, 7, 6, 11, 13, 21, 14, 1 \rangle$, tutti i valori c_i relativi sono contenuti nel vettore presentato in Figura 3.

i	0	1	2	3	4	5	6	7	8	9
x_i	ε	2	4	7	6	11	13	21	14	1
	0	1	2	3	3	4	5	6	6	1

Figura 3: Sottosequenza crescente: vettore di tutti i c_i ottenuti considerando la sequenza $X = \langle 2, 4, 7, 6, 11, 13, 21, 14, 1 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

3.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 3.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione c_i di ogni sottoproblema. L'algoritmo 5 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LGSRIC}(m)$ (si suppone di aver accesso alla sequenza X). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 5 Lunghezza di una più lunga sottosequenza crescente di una sequenza X di lunghezza m (tecnica: ricorsione).

```

procedure LGSRIC( $i$ )
  if  $i = 1$  then
    return 1
  else
     $max \leftarrow 0$ 
    for  $h \leftarrow 1$  to  $i - 1$  do
      if  $x_h < x_i$  then
         $S \leftarrow \text{LGSRIC}(h)$ 
        if  $S > max$  then
           $max \leftarrow S$ 
    return  $1 + max$ 

```

3.7 Implementazione bottom-up

L'algoritmo 6 presenta lo pseudocodice che calcola tutti i valori $c_0, c_1, \dots, c_m$ basandosi sulle equazioni di ricorrenza definite nella Sezione 3.4. Aniché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m^2)$ occupando $\Theta(m)$ spazio in memoria (ossia un vettore che contiene i vari c_i).

Algorithm 6 Lunghezza di una più lunga sottosequenza crescente di una sequenza X di lunghezza m (tecnica: bottom-up).

```
procedure LGS( $X$ )  
   $c[1] \leftarrow 1$   
   $max \leftarrow c[1]$   
  for  $i \leftarrow 2$  to  $m$  do  
     $temp \leftarrow 0$   
    for  $h \leftarrow 1$  to  $i - 1$  do  
      if  $(x_h < x_i) \wedge (c[h] > temp)$  then  
         $temp \leftarrow c[h]$   
     $c[i] \leftarrow 1 + temp$   
    if  $c[i] > max$  then  
       $max \leftarrow c[i]$   
return  $max$ 
```

4 Più lunga sottosequenza zig-zag (LZS)

4.1 Definizione del problema

Il problema PB è definito come segue:

data una sequenza X di m numeri interi, si determini UNA tra le più lunghe sottosequenze zig-zag di X

Una sequenza $X = \langle x_1, x_2, \dots, x_m \rangle$ è detta *zig-zag* se, per ogni $i \in \{1, 2, \dots, m\}$ vale:

$$\begin{aligned} x_i &< x_{i+1} && \text{se } i \text{ è dispari} \\ x_i &> x_{i+1} && \text{se } i \text{ è pari} \end{aligned} \tag{1}$$

Ad esempio, data $X = \langle 3, 4, 8, 5, 6, 2 \rangle$, una tra le più lunghe sottosequenze zig-zag di X è la sottosequenza $\langle 3, 8, 5, 6, 2 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze zig-zag di X .

Ad esempio, data $X = \langle 3, 4, 8, 5, 6, 2 \rangle$, la lunghezza di una tra le più lunghe sottosequenze zig-zag di X è 5.

4.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la sequenza X ma un suo prefisso.

Il **sottoproblema di dimensione** (i) è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze zig-zag di X_i

Dato che $0 \leq i \leq m$, si ottengono $(m + 1)$ sottoproblemi (i può valere 0 in quanto si deve considerare anche il caso in cui il prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i) , la variabile ad esso associata è c_i ed è così definita:

$$c_i = \text{lunghezza di una tra le più lunghe sottosequenze zig-zag di } X_i$$

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione (3) , il valore della variabile c_3 si otterrà utilizzando le variabili c_0 , c_1 e c_2 . Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_0, \dots, c_{i-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione c_i andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i) ed, eventualmente, accodando l'elemento x_i estratto dall'input X_i ?

Purtroppo, però, il problema così definito non è risolvibile: **ci manca informazione**. Date solamente le variabili $\{c_0, \dots, c_{i-1}\}$ e il prefisso X_i (del quale, in realtà, ci interessa solo l'elemento x_i), non c'è alcun modo per poter comprendere se l'elemento x_i possa essere accodato alle sottosequenze zig-zag relative ai sottoproblemi di dimensione minore a i : non sappiamo con quale elemento termini ognuna di queste sottosequenze ma ne conosciamo solo la lunghezza.

Risulta necessario introdurre un problema ausiliario, nel quale introdurre l'informazione mancante necessaria.

4.3 Problema ausiliario P' di PBR

Il problema ausiliario P' è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze zig-zag di X e che termina con x_m

In altre parole, si richiede che la sottosequenza zig-zag più lunga termini con l'ultimo elemento della sequenza in input.

Come il problema ridotto PBR, anche il problema ausiliario contiene $m+1$ diversi sottoproblemi, ognuno associato ad una differente variabile. Il sottoproblema di dimensione (i) è definito come:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze zig-zag di X_i e che termina con x_i

ed è associato alla variabile c_i così definita:

$$c_i = \text{lunghezza di una tra le più lunghe sottosequenze zig-zag di } X_i \text{ e che termina con } x_i$$

Si noti quindi che per un qualunque sottoproblema di dimensione (s) , solo imponendo che la soluzione c_s si riferisca ad una sottosequenza zig-zag di X_s che termina con x_s è possibile stabilire se un altro elemento x_u con $u > s$ possa essere accodato a tale sottosequenza. Per poter sapere, però, se l'elemento x_u possa essere accodato alla soluzione di un sottoproblema di dimensione minore, è necessario sapere se la lunghezza della soluzione è pari o dispari:

- nel caso la soluzione avesse lunghezza dispari, allora l'elemento x_u può essere accodato solo se è maggiore dell'ultimo elemento della soluzione
- nel caso la soluzione avesse lunghezza pari, allora l'elemento x_u può essere accodato solo se è minore dell'ultimo elemento della soluzione

Per questa ragione, per ogni sottoproblema di dimensione (i) bisogna andare a definire due variabili:

$c_{i,0}$ = lunghezza di una tra le più lunghe sottosequenze zig-zag di lunghezza pari di X_i e che termina con x_i
 $c_{i,1}$ = lunghezza di una tra le più lunghe sottosequenze zig-zag di lunghezza dispari di X_i e che termina con x_i

4.4 Equazioni di ricorrenza

È ora possibile risolvere il problema ausiliario P' andando a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

4.4.1 Caso base: (i) con $i = 1$

Il caso base si ha per un qualunque sottoproblema di dimensione (i) con $i = 1$, ossia quando il prefisso considerato è una sequenza composta da un singolo elemento. In questo caso, è facile ottenere il valore delle variabili $c_{i,0}$ e $c_{i,1}$: la lunghezza di una tra le più lunghe sottosequenze zig-zag di una sequenza composta da un singolo elemento e che termina con quell'elemento è 1 (dispari). Pertanto, nel caso di sottosequenza zig-zag di lunghezza pari (ossia $c_{1,0}$), la soluzione deve essere 0; nel caso di sottosequenza zig-zag di lunghezza dispari (ossia $c_{1,1}$), invece, la soluzione deve essere 1. Dunque, il caso base è scrivibile come:

$$\begin{aligned}
 c_{i,0} &= 0 & \text{se } i &= 1 \\
 c_{i,1} &= 1 & \text{se } i &= 1
 \end{aligned} \tag{2}$$

4.4.2 Passo ricorsivo: (i) con $i > 1$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i) con $i > 1$, ossia quando si considera un prefisso della sequenza X in input di almeno due elementi. I dati disponibili per calcolare $c_{i,0}$ e $c_{i,1}$ sono: l'input X ed in particolare l'elemento x_i e tutte le variabili $\{c_{0,0}, \dots, c_{i-1,0}\}$ e $\{c_{0,1}, \dots, c_{i-1,1}\}$.

Iniziamo col definire come poter calcolare il valore di $c_{i,1}$. Si ricorda che i valori $c_{1,0}, c_{2,0}, \dots, c_{i-1,0}$ rappresentano le lunghezze delle più lunghe sottosequenze zig-zag di lunghezza pari di $X_1, X_2, \dots, X_{i-1}$ che terminano, rispettivamente, con $x_1, x_2, \dots, x_{i-1}$. Tra queste, ci saranno alcune sottosequenze alle quali possiamo accodare x_i (in quanto minore dell'ultimo elemento) e altre alle quali

l'elemento x_i non può essere accodato (in quanto maggiore dell'ultimo elemento). Se prendiamo la più lunga sottosequenza zig-zag di lunghezza pari alla quale possiamo attaccare x_i e accodiamo ad essa x_i , otteniamo una più lunga sottosequenza zig-zag di lunghezza dispari di X_i che termina con x_i (ossia otteniamo il valore di $c_{i,1}$). La lunghezza di tale sottosequenza, infatti, è uguale alla lunghezza della sottosequenza zig-zag alla quale abbiamo accodato x_i aumentata di 1. Nel caso in cui x_i non possa essere accodato ad alcuna sottosequenza, la soluzione $c_{i,1}$ del sottoproblema deve essere posta a 1: la più lunga sottosequenza zig-zag di lunghezza dispari di X_i che termina con x_i è la sequenza $\langle x_i \rangle$ (che ha lunghezza 1, dispari). Questo caso può essere riscritto come segue, dato $i > 1$:

$$c_{i,1} = \begin{cases} 1 + \max\{c_{k,0} \mid 1 \leq k < i, x_k > x_i\} & \text{se } \exists k : 1 \leq k < i, x_k > x_i \\ 1 & \text{altrimenti} \end{cases}$$

Definiamo ora come poter calcolare il valore di $c_{i,0}$. Si ricorda che i valori $c_{1,1}$, $c_{2,1}$, $\dots$, $c_{i-1,1}$ rappresentano le lunghezze delle più lunghe sottosequenze zig-zag di lunghezza dispari di $X_1, X_2, \dots, X_{i-1}$ che terminano, rispettivamente, con $x_1, x_2, \dots, x_{i-1}$. Tra queste, ci saranno alcune sottosequenze alle quali possiamo accodare x_i (in quanto maggiore dell'ultimo elemento) e altre alle quali l'elemento x_i non può essere accodato (in quanto minore dell'ultimo elemento). Se prendiamo la più lunga sottosequenza zig-zag di lunghezza dispari alla quale possiamo attaccare x_i e accodiamo ad essa x_i , otteniamo una più lunga sottosequenza zig-zag di lunghezza pari di X_i che termina con x_i . La lunghezza di tale sottosequenza, infatti, è uguale alla lunghezza della sottosequenza zig-zag alla quale abbiamo accodato x_i aumentata di 1. Nel caso in cui x_i non possa essere accodato ad alcuna sottosequenza, la soluzione $c_{i,1}$ del sottoproblema deve essere posta a 0: la più lunga sottosequenza zig-zag di lunghezza pari di X_i che termina con x_i dovrebbe essere la sequenza $\langle x_i \rangle$ che ha però lunghezza 1, dispari. Questo caso può essere riscritto come segue, dato $i > 1$:

$$c_{i,0} = \begin{cases} 1 + \max\{c_{k,1} \mid 1 \leq k < i, x_k < x_i\} & \text{se } \exists k : 1 \leq k < i, x_k < x_i \\ 0 & \text{altrimenti} \end{cases}$$

4.5 Soluzione del problema

Una volta calcolati i valori $c_{0,0}, c_{1,0}, \dots, c_{m,0}$ e $c_{0,1}, c_{1,1}, \dots, c_{m,1}$ si hanno a disposizione tutte le lunghezze di una tra le più lunghe sottosequenze zig-zag, sia di lunghezza pari che di lunghezza dispari, dei vari prefissi di X e che terminano con l'ultimo elemento del prefisso. La soluzione del problema si ottiene calcolando:

$$\max\{c_{i,0}, c_{i,1} \mid 1 \leq i \leq m\}$$

Esempio Data $X = \langle 3, 4, 8, 5, 6, 2 \rangle$, tutti i valori $c_{i,0}$ e $c_{i,1}$ relativi sono contenuti nel vettore presentato in Figura 4.

4.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 4.4, è immediato ricavare l'algoritmo ricorsivo per determinare le soluzioni $c_{i,0}$ e $c_{i,1}$ di ogni

i	1	2	3	4	5	6
x_i	3	4	8	5	6	2
$c_{i,0}$	0	2	2	2	4	0
$c_{i,1}$	1	1	1	3	3	5

Figura 4: Sottosequenza zig-zag: vettore di tutti i $c_{i,0}$ e $c_{i,1}$ ottenuti considerando la sequenza $X = \langle 3, 4, 8, 5, 6, 2 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

sottoproblema. L'algoritmo 7 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LZSRIC}(m)$ (si suppone di aver accesso alla sequenza X). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

4.7 Implementazione bottom-up

L'algoritmo 8 presenta lo pseudocodice che calcola tutti i valori $c_{0,0}, c_{1,0}, \dots, c_{m,0}$ e $c_{0,1}, c_{1,1}, \dots, c_{m,1}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 4.4. Aniché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m^2)$ occupando $\Theta(2 \cdot m)$ spazio in memoria (ossia due vettore che contengono le varie soluzioni, uno per i $c_{i,0}$ e l'altro per i $c_{i,1}$).

Algorithm 7 Lunghezza di una più lunga sottosequenza zig-zag di una sequenza X di lunghezza m (tecnica: ricorsione).

```

procedure LZSRIC( $n$ )
     $max \leftarrow 0$ 
    for  $i \leftarrow 1$  to  $n$  do
         $m \leftarrow \text{MAX}(\text{LZSric}_0(i), \text{LZSric}_1(i))$ 
        if  $m > max$  then
             $max \leftarrow m$ 
    return  $max$ 

procedure LZSRIC_0( $i$ )
    if  $i = 1$  then
        return 0
    else
         $max \leftarrow 0$ 
         $flag \leftarrow false$ 
        for  $h \leftarrow 1$  to  $i - 1$  do
             $S \leftarrow \text{LZSRIC}_1(h)$ 
            if  $(x_h < x_i) \wedge S > max$  then
                 $max \leftarrow S$ 
                 $flag \leftarrow true$ 
        if  $flag$  then
            return  $1 + max$ 
        else
            return 0

procedure LZSRIC_1( $i$ )
    if  $i = 1$  then
        return 1
    else
         $max \leftarrow 0$ 
        for  $h \leftarrow 1$  to  $i - 1$  do
             $S \leftarrow \text{LZSRIC}_0(h)$ 
            if  $(x_h > x_i) \wedge S > max$  then
                 $max \leftarrow S$ 
        return  $1 + max$ 

```

▷ Ricorderà se h si è trovato

Algorithm 8 Lunghezza di una più lunga sottosequenza zig-zag di una sequenza X di lunghezza m (tecnica: bottom-up).

procedure LZS(X)

$c[1, 0] \leftarrow 0$

$c[1, 1] \leftarrow 1$

$max \leftarrow 0$

for $i \leftarrow 2$ **to** m **do**

$max \leftarrow 0$

for $h \leftarrow 1$ **to** $i - 1$ **do**

if $(x_h > x_i) \wedge c[h, 0] > max$ **then**

$max \leftarrow c[h, 0]$

$c[i, 1] \leftarrow 1 + max$

$max \leftarrow 0$

$flag \leftarrow false$

$\triangleright$ Ricorderà se h si è trovato

for $h \leftarrow 1$ **to** $i - 1$ **do**

if $(x_h < x_i) \wedge c[h, 1] > max$ **then**

$max \leftarrow c[h, 1]$

$flag \leftarrow true$

if $flag$ **then**

$c[i, 0] \leftarrow 1 + max$

else

$c[i, 0] \leftarrow 0$

$max \leftarrow 0$

for $i \leftarrow 1$ **to** m **do**

$m \leftarrow \text{MAX}(c[i, 0], c[i, 1])$

if $m > max$ **then**

$max \leftarrow m$

return m

5 Più lunga sottosequenza comune crescente (LGCS)

5.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m e n numeri interi, si determini
UNA *tra le più lunghe sottosequenze crescenti comuni a X e Y*

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, una tra le più lunghe sottosequenze crescenti comuni è la sottosequenza $\langle 2, 4, 21 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti comuni a X e Y

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, la lunghezza di una tra le più lunghe sottosequenze crescenti comuni è 3.

5.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la coppia (X, Y) ma una coppia di prefissi di tali sequenze.

Il **sottoproblema di dimensione** (i, j) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti comuni al prefisso X_i e al prefisso Y_j

Dato che $0 \leq i \leq m$ e $0 \leq j \leq n$, si ottengono $(m+1) \cdot (n+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j) , la variabile ad esso associata è $c_{i,j}$ ed è così definita:

$c_{i,j} =$ *lunghezza di una tra le più lunghe sottosequenze crescenti comuni a X_i e Y_j*

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(2, 2)$, il valore della variabile $c_{2,2}$ si otterrà utilizzando le variabili $c_{0,0}$, $c_{0,1}$, $c_{1,0}$, $c_{1,1}$, $c_{1,2}$ e $c_{2,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di

poter utilizzare le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i,j) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) ?

Purtroppo, però, il problema così definito non è risolvibile: **ci manca informazione**. Date solamente le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$ e i prefissi X_i e Y_j (dei quali, in realtà, ci interessano solo gli elementi x_i e y_j), non c'è alcun modo per poter comprendere se gli elementi x_i e y_j , nel caso fossero uguali, possano essere accodati alle sottosequenze crescenti relative ai sottoproblemi di dimensione minore a (i,j) : non sappiamo con quale elemento terminare ognuna di queste sottosequenze ma ne conosciamo solo la lunghezza.

Risulta necessario introdurre un problema ausiliario, nel quale introdurre l'informazione mancante necessaria.

5.3 Problema ausiliario P' di PBR

Il problema ausiliario P' è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti comuni a X e a Y e che termina con x_m e y_n (se questi coincidono)

In altre parole, si richiede che la sottosequenza soluzione del problema termini con l'ultimo elemento delle sequenze X e Y nel caso siano lo stesso elemento (in caso contrario, la soluzione è la sequenza vuota).

Come il problema ridotto PBR, anche il problema ausiliario contiene $(m+1) \cdot (n+1)$ diversi sottoproblemi, ognuno associato ad una differente variabile. Il sottoproblema di dimensione (i,j) è definito come:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze crescenti comuni a X_i e a Y_j e che termina con x_i e y_j (se questi coincidono)

ed è associato alla variabile $c_{i,j}$ così definita:

$c_{i,j}$ = *lunghezza di una tra le più lunghe sottosequenze crescenti comuni a X_i e Y_j e che termina con x_i e y_j (se questi coincidono)*

Si noti quindi che per un qualunque sottoproblema di dimensione (s,t) , solo imponendo che la soluzione $c_{s,t}$ si riferisca ad una sottosequenza crescente comune a X_s e Y_t che termina con $x_s = y_t$ è possibile stabilire se un altro elemento $x_u = y_v$ con $u > s$ e $v > t$ possa essere accodato a tale sottosequenza (andando a verificare che $x_s = y_t \leq x_u = y_v$).

5.4 Equazioni di ricorrenza

È ora possibile risolvere il problema ausiliario P' andando a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

5.4.1 Caso base

Certamente fanno parte del caso base tutte le coppie (i, j) con $i = 0 \vee j = 0$ ma il caso base si ha anche per un qualunque sottoproblema di dimensione (i, j) tale che $x_i \neq y_j$, ossia quando i due prefissi X_i e Y_j considerati terminano con due elementi diversi. Questo caso, infatti, è semplice da risolvere: per definizione di $c_{i,j}$ gli elementi x_i e y_j devono coincidere, in caso contrario la soluzione è 0 (ossia la lunghezza della sequenza vuota). Pertanto, si può evitare di considerare tutte le coppie (i, j) con $i = 0 \vee j = 0$: il numero di sottoproblemi che compone il problema ausiliario P' possono essere ridotti a $m \cdot n$.

Riassumendo, il caso base si ha per quei

sottoproblemi di dimensione (i, j) con $i > 0 \wedge j > 0$ tali che $x_i \neq y_j$

ed è scrivibile come

$$c_{i,j} = 0$$

5.4.2 Passo ricorsivo: (i, j) con $i > 0$ e $j > 0$ tali che $x_i = y_j$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, j) tale che $x_i = y_j$, ossia quando i due prefissi X_i e Y_j considerati terminano con lo stesso elemento. In questo caso, la lunghezza della più lunga sottosequenza crescente comune fra X_i e Y_j è uguale alla lunghezza della più lunga sottosequenza crescente comune calcolata per un sottoproblema di dimensione minore e che termina con un carattere minore di $x_i = y_j$ aumentata di uno (in quanto si accoda l'elemento comune $x_i = y_j$ alla sottosequenza crescente comune relativa al sottoproblema considerato). Il tutto può essere scritto come:

$$c_{i,j} = 1 + \max\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, x_h < x_i\}$$

Poiché può accadere che l'insieme $\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, x_h < x_i\}$ sia vuoto (il che corrisponde al fatto che l'elemento $x_i = y_j$ è minore di tutti gli elementi precedenti e, quindi, non può essere accodato a nessuna più lunga sottosequenza crescente comune relativa a sottoproblemi di dimensione minore), assumiamo per definizione che $\max\{\emptyset\} = 0$, così che il corrispondente valore di $c_{i,j}$ risulti uguale a 1. Inoltre, si noti che l'insieme $\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, x_h < x_i\}$ corrisponde all'insieme vuoto anche se $i = 1 \vee j = 1$ in quanto i casi con $i = 0 \vee j = 0$ sono stati omessi dal caso base e, pertanto, non esistono sottoproblemi di dimensione minore a quella del sottoproblema considerato (ossia quello con $i = 1 \vee j = 1$).

		1	2	3	4	5	6	7	8	<i>j</i>
		2	7	4	23	21	14	1	8	<i>y_j</i>
1	2	1	0	0	0	0	0	0	0	
2	4	0	0	2	0	0	0	0	0	
3	7	0	2	0	0	0	0	0	0	
4	11	0	0	0	0	0	0	0	0	
5	21	0	0	0	0	3	0	0	0	
6	14	0	0	0	0	0	3	0	0	
7	1	0	0	0	0	0	0	1	0	
<i>i</i>	<i>x_i</i>									

Figura 5: Sottosequenza crescente comune: matrice di tutti i $c_{i,j}$ ottenuti considerando le sequenze $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

5.5 Soluzione del problema

Una volta calcolati i valori $c_{1,1}, c_{2,1}, c_{1,2}, \dots, c_{m,n}$ si hanno a disposizione tutte le lunghezze delle sottosequenze comuni massimali fra qualsiasi prefisso di X e qualsiasi prefisso di Y che non hanno elementi consecutivi dello stesso colore e che terminano con l'ultimo elemento di entrambi i prefissi. La soluzione al problema ausiliario P' è $c_{m,n}$ mentre quella del problema ridotto PBR è:

$$\max\{c_{i,j} \mid 1 \leq i \leq m \wedge 1 \leq j \leq n\}$$

Esempio Date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, tutti i valori $c_{i,j}$ relativi sono contenuti nella matrice presentata in Figura 5.

5.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 5.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j}$ di ogni sottoproblema. L'algoritmo 9 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LGCSRIC}(m, n)$ (si suppone di aver accesso alle sequenze X e Y). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 9 Lunghezza di una più lunga sottosequenza crescente comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: ricorsione).

```

procedure LGCSRIC( $i, j$ )
  if  $i = 0 \vee j = 0 \vee x_i \neq y_j$  then
    return 0
  else
     $max \leftarrow 0$ 
    for  $h \leftarrow 1$  to  $i - 1$  do
      for  $k \leftarrow 1$  to  $j - 1$  do
        if  $x_h < x_i$  then
           $S \leftarrow \text{LGCSRIC}(h, k)$ 
          if  $S > max$  then
             $max \leftarrow S$ 
    return  $1 + max$ 

```

5.7 Implementazione bottom-up

L'algoritmo 10 presenta lo pseudocodice che calcola tutti i valori $c_{0,0}, c_{0,1}, \dots, c_{m,n}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 5.4. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m^2 \cdot n^2)$ occupando $\Theta(m \cdot n)$ spazio in memoria (ossia una matrice che contiene i vari $c_{i,j}$).

Algorithm 10 Lunghezza di una più lunga sottosequenza crescente comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: bottom-up).

```

procedure LGCS( $X, Y$ )
   $max \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      if  $x_i \neq y_j$  then
         $c[i, j] \leftarrow 0$ 
      else
         $temp \leftarrow 0$ 
        for  $h \leftarrow 1$  to  $i - 1$  do
          for  $k \leftarrow 1$  to  $j - 1$  do
            if  $(x_h < x_i) \wedge (c[h, k] > temp)$  then
               $temp \leftarrow c[h, k]$ 
         $c[i, j] \leftarrow 1 + temp$ 
      if  $c[i, j] > max$  then
         $max \leftarrow c[i, j]$ 
  return  $max$ 

```

6 Più lunga sottosequenza con colori alternanti (LAS)

Premessa Sia C un insieme di colori. Gli elementi della sequenza considerata in questo esercizio sono numeri naturali colorati. La funzione di colorazione è definita nel seguente modo: $\phi : \mathbb{N} \rightarrow C$. Per esempio, data la funzione:

$$\phi(x) = \begin{cases} \text{rosso} & x < 5 \\ \text{blu} & 5 \leq x \leq 10 \\ \text{verde} & x > 10 \end{cases}$$

la sequenza $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 1 \rangle$ sarà colorata nel seguente modo: $\langle r, r, b, b, g, r, g, g, r \rangle$.

6.1 Definizione del problema

Il problema PB è definito come segue:

data una sequenza X di m numeri interi, si determini UNA tra le più lunghe sottosequenze di X che non abbia elementi consecutivi dello stesso colore

Ad esempio, data $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 12 \rangle$ colorata utilizzando la funzione ϕ definita sopra, una tra le più lunghe sottosequenze di X che non ha elementi consecutivi dello stesso colore è la sottosequenza $\langle 2, 7, 11, 3, 21 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze di X che non abbia elementi consecutivi dello stesso colore

Ad esempio, data $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 12 \rangle$, la lunghezza di una tra le più lunghe sottosequenze di X che non ha elementi consecutivi dello stesso colore è 5.

6.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la sequenza X ma un suo prefisso.

Il **sottoproblema di dimensione** (i) è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze di X_i che non abbia elementi consecutivi dello stesso colore

Dato che $0 \leq i \leq m$, si ottengono $(m + 1)$ sottoproblemi (i può valere 0 in quanto si deve considerare anche il caso in cui il prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i), la variabile ad esso associata è c_i ed è così definita:

$c_i =$ lunghezza di una tra le più lunghe sottosequenze di X_i che non ha elementi consecutivi dello stesso colore

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione (3) , il valore della variabile c_3 si otterrà utilizzando le variabili c_0 , c_1 e c_2 . Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_0, \dots, c_{i-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione c_i andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i) ed, eventualmente, accodando l'elemento x_i estratto dall'input X_i ?

Purtroppo, però, il problema così definito non è risolvibile: **ci manca informazione**. Date solamente le variabili $\{c_0, \dots, c_{i-1}\}$ e il prefisso X_i (del quale, in realtà, ci interessa solo l'elemento x_i), non c'è alcun modo per poter comprendere se l'elemento x_i possa essere accodato alle sottosequenze senza elementi consecutivi dello stesso colore relative ai sottoproblemi di dimensione minore a i : non sappiamo con quale elemento (e quindi con quale colore) termini ognuna di queste sottosequenze ma ne conosciamo solo la lunghezza.

Risulta necessario introdurre un problema ausiliario, nel quale introdurre l'informazione mancante necessaria.

6.3 Problema ausiliario P' di PBR

Il problema ausiliario P' è definito come segue:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze di X che non abbia elementi consecutivi dello stesso colore e che termini con x_m

In altre parole, si richiede che la sottosequenza più lunga senza elementi consecutivi dello stesso colore termini con l'ultimo elemento della sequenza in input.

Come il problema ridotto PBR, anche il problema ausiliario contiene $m+1$ diversi sottoproblemi, ognuno associato ad una differente variabile. Il sottoproblema di dimensione (i) è definito come:

data una sequenza X di m numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze di X_i che non abbia elementi consecutivi dello stesso colore e che termini con x_i

ed è associato alla variabile c_i così definita:

$c_i =$ lunghezza di una tra le più lunghe sottosequenze di X_i che non ha elementi consecutivi dello stesso colore e che termina con x_i

Si noti quindi che per un qualunque sottoproblema di dimensione (s), solo imponendo che la soluzione c_s si riferisca ad una sottosequenza di X_s che non ha elementi consecutivi dello stesso colore e che termina con x_s è possibile stabilire se un altro elemento x_u con $u > s$ possa essere accodato a tale sottosequenza (andando a verificare che $\phi(x_s) \neq \phi(x_u)$).

6.4 Equazioni di ricorrenza

É ora possibile risolvere il problema ausiliario P' andando a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

6.4.1 Caso base: (i) con $i \leq 1$

Il caso base si ha per un qualunque sottoproblema di dimensione (i) con $i = 0 \vee i = 1$, ossia quando il prefisso considerato è la sequenza vuota oppure è una sequenza composta da un singolo elemento. In entrambi i casi, è facile ottenere il valore della variabile c_i : la lunghezza di una tra le più lunghe sottosequenze della sequenza vuota che non ha elementi consecutivi dello stesso colore è 0 (la lunghezza della sequenza vuota) mentre la lunghezza di una tra le più lunghe sottosequenze di una sequenza di un singolo elemento che non ha elementi consecutivi dello stesso colore e che termina con quell'elemento è 1. Il caso base $i = 0$, comunque, risulta non necessario in quanto è possibile utilizzare, come caso base, solamente il caso $i = 1$. Per questa ragione, il caso base è scrivibile come:

$$c_i = 1 \quad \text{se } i = 1$$

6.4.2 Passo ricorsivo: (i) con $i > 1$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i) con $i > 1$, ossia quando si considera un prefisso della sequenza X in input di almeno due elementi. I dati disponibili per calcolare c_i sono: l'input X ed in particolare l'elemento x_i e tutte le variabili $\{c_0, \dots, c_{i-1}\}$. Si ricorda che i valori $c_1, c_2, \dots, c_{i-1}$ rappresentano le lunghezze delle più lunghe sottosequenze di $X_1, X_2, \dots, X_{i-1}$ che non contengono elementi consecutivi dello stesso colore e che terminano, rispettivamente, con $x_1, x_2, \dots, x_{i-1}$. Tra queste ci saranno alcune sottosequenze alle quali possiamo accodare x_i (in quanto colorato in modo diverso dell'ultimo elemento) e altre alle quali l'elemento x_i non può essere accodato (in quanto colorato allo stesso modo dell'ultimo elemento). Se prendiamo la più lunga sottosequenza alla quale possiamo attaccare x_i e accodiamo

<i>i</i>	0	1	2	3	4	5	6	7	8	9
<i>x_i</i>	ε	2	4	7	6	11	3	21	14	12
<i>φ(x_i)</i>	-	<i>r</i>	<i>r</i>	<i>b</i>	<i>b</i>	<i>g</i>	<i>r</i>	<i>g</i>	<i>g</i>	<i>g</i>
	0	1	1	2	2	3	4	5	5	5

Figura 6: Sottosequenza con colori alternanti: vettore di tutti i c_i ottenuti considerando la sequenza $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 12 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

ad essa x_i , otteniamo la più lunga sottosequenza di X_i che non contiene elementi consecutivi dello stesso colore e che termina con x_i . La lunghezza di tale sottosequenza, pertanto, sarà uguale alla lunghezza della sottosequenza alla quale abbiamo accodato x_i aumentata di 1. Il passo ricorsivo è quindi scrivibile come:

$$c_i = 1 + \max\{c_h \mid 1 \leq h < i \wedge \phi(x_h) \neq \phi(x_i)\}$$

Poiché può accadere che l'insieme $\{c_h \mid 1 \leq h < i \wedge \phi(x_h) \neq \phi(x_i)\}$ sia vuoto (il che corrisponde al fatto che l'elemento x_i è colorato allo stesso modo di tutti gli elementi precedenti e, quindi, non può essere accodato a nessuna più lunga sottosequenza alternante relativa a sottoproblemi di dimensione minore), assumiamo per definizione che $\max\{\emptyset\} = 0$, così che il corrispondente valore di c_i risulti uguale a 1.

6.5 Soluzione del problema

Una volta calcolati i valori $c_0, c_1, \dots, c_m$ si hanno a disposizione le lunghezze di una tra le più lunghe sottosequenze dei vari prefissi della sequenza in input che non contengono elementi consecutivi dello stesso colore e che terminano con l'ultimo elemento del prefisso. La soluzione al problema ausiliario P' è c_m mentre quella del problema ridotto PBR è

$$\max\{c_i \mid 1 \leq i \leq m\}$$

Esempio Data $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 12 \rangle$, tutti i valori c_i relativi sono contenuti nel vettore presentato in Figura 6.

6.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 6.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione c_i di ogni sottoproblema. L'algoritmo 11 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LASRIC}(m)$ (si suppone di aver accesso alla sequenza X). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene

un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 11 Lunghezza di una più lunga sottosequenza di una sequenza X lunga m che non presenta elementi consecutivi dello stesso colore (tecnica: ricorsione).

```

procedure LASRIC( $i$ )
  if  $i = 1$  then
    return 1
  else
     $max \leftarrow 0$ 
    for  $h \leftarrow 1$  to  $i - 1$  do
      if  $\phi(x_h) \neq \phi(x_i)$  then
         $S \leftarrow \text{LASRIC}(h)$ 
        if  $S > max$  then
           $max \leftarrow S$ 
    return  $1 + max$ 

```

6.7 Implementazione bottom-up

L'algoritmo 12 presenta lo pseudocodice che calcola tutti i valori $c_0, c_1, \dots, c_m$ basandosi sulle equazioni di ricorrenza definite nella Sezione 6.4. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m^2)$ occupando $\Theta(m)$ spazio in memoria (ossia un vettore che contiene i vari c_i).

Algorithm 12 Lunghezza di una più lunga sottosequenza di una sequenza X lunga m che non presenta elementi consecutivi dello stesso colore (tecnica: bottom-up).

```

procedure LAS( $X$ )
   $c[1] \leftarrow 1$ 
   $max \leftarrow c[1]$ 
  for  $i \leftarrow 2$  to  $m$  do
     $temp \leftarrow 0$ 
    for  $h \leftarrow 1$  to  $i - 1$  do
      if  $(\phi(x_h) \neq \phi(x_i)) \wedge (c[h] > temp)$  then
         $temp \leftarrow c[h]$ 
     $c[i] \leftarrow 1 + temp$ 
    if  $c[i] > max$  then
       $max \leftarrow c[i]$ 
  return  $max$ 

```

7 Più lunga sottosequenza comune con colore alternante (LACS)

Premessa Sia C un insieme di colori. Gli elementi della sequenza considerata in questo esercizio sono numeri naturali colorati. La funzione di colorazione è definita nel seguente modo: $\phi : \mathbb{N} \rightarrow C$. Per esempio, data la funzione:

$$\phi(x) = \begin{cases} \text{rosso} & x < 5 \\ \text{blu} & 5 \leq x \leq 10 \\ \text{verde} & x > 10 \end{cases}$$

la sequenza $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 1 \rangle$ sarà colorata nel seguente modo: $\langle r, r, b, b, g, r, g, g, r \rangle$.

7.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m e n numeri interi, si determini UNA tra le più lunghe sottosequenze comuni a X e Y che non abbia elementi consecutivi dello stesso colore

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, una tra le più lunghe sottosequenze comuni senza elementi consecutivi dello stesso colore è la sottosequenza $\langle 2, 7, 21, 1 \rangle$.

Quello che si andrà a risolvere, comunque, è una versione ridotta del problema PB. La versione ridotta PBR è definita come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X e Y che non abbia elementi consecutivi dello stesso colore

Ad esempio, date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, la lunghezza di una tra le più lunghe sottosequenze comuni senza elementi consecutivi dello stesso colore è 4.

7.2 Sottoproblemi e variabili associate

Il problema PBR contiene diversi sottoproblemi ognuno dei quali non ha come input la coppia (X, Y) ma una coppia di prefissi di tali sequenze.

Il **sottoproblema di dimensione** (i, j) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j che non abbia elementi consecutivi dello stesso colore

Dato che $0 \leq i \leq m$ e $0 \leq j \leq n$, si ottengono $(m+1) \cdot (n+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j) , la variabile ad esso associata è $c_{i,j}$ ed è così definita:

$c_{i,j}$ = lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j che non ha elementi consecutivi dello stesso colore

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(2, 2)$, il valore della variabile $c_{2,2}$ si otterrà utilizzando le variabili $c_{0,0}$, $c_{0,1}$, $c_{1,0}$, $c_{1,1}$, $c_{1,2}$ e $c_{2,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) ?

Purtroppo, però, il problema così definito non è risolvibile: **ci manca informazione**. Date solamente le variabili $\{c_{0,0}, \dots, c_{i-1,j}, c_{i,j-1}\}$ e i prefissi X_i e Y_j (dei quali, in realtà, ci interessano solo gli elementi x_i e y_j), non c'è alcun modo per poter comprendere se gli elementi x_i e y_j , nel caso fossero uguali, possano essere accodati alle sottosequenze comuni alternanti relative ai sottoproblemi di dimensione minore a (i, j) : non sappiamo con quale elemento (e quindi con quale colore) terminare ognuna di queste sottosequenze ma ne conosciamo solo la lunghezza.

Risulta necessario introdurre un problema ausiliario, nel quale introdurre l'informazione mancante necessaria.

7.3 Problema ausiliario P' di PBR

Il problema ausiliario P' è definito come segue:

*date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X e Y che non abbia elementi consecutivi dello stesso colore
e che termini con x_m e y_n (se questi coincidono)*

In altre parole, si richiede che la sottosequenza soluzione del problema termini con l'ultimo elemento delle sequenze X e Y nel caso siano lo stesso elemento (in caso contrario, la soluzione è la sequenza vuota).

Come il problema ridotto PBR, anche il problema ausiliario contiene $(m + 1) \cdot (n + 1)$ diversi sottoproblemi, ognuno associato ad una differente variabile. Il sottoproblema di dimensione (i, j) è definito come:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, si determini la lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j che non abbia elementi consecutivi dello stesso colore e che termini con x_i e y_j (se questi coincidono)

ed è associato alla variabile $c_{i,j}$ così definita:

$c_{i,j}$ = lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j che non ha elementi consecutivi dello stesso colore e che termina con x_i e y_j (se questi coincidono)

Si noti quindi che per un qualunque sottoproblema di dimensione (s, t) , solo imponendo che la soluzione $c_{s,t}$ si riferisca ad una sottosequenza comune a X_s e Y_t che non presenta elementi consecutivi dello stesso colore e che termina con $x_s = y_t$ è possibile stabilire se un altro elemento $x_u = y_v$ con $u > s$ e $v > t$ possa essere accodato a tale sottosequenza (andando a verificare che $\phi(x_s) = \phi(y_t) \neq \phi(x_u) = \phi(y_v)$).

7.4 Equazioni di ricorrenza

É ora possibile risolvere il problema ausiliario P' andando a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

7.4.1 Caso base

Certamente fanno parte del caso base tutte le coppie (i, j) con $i = 0 \vee j = 0$ ma il caso base si ha anche per un qualunque sottoproblema di dimensione (i, j) tale che $x_i \neq y_j$, ossia quando i due prefissi X_i e Y_j considerati terminano con due elementi diversi. Questo caso, infatti, è semplice da risolvere: per definizione di $c_{i,j}$ gli elementi x_i e y_j devono coincidere, in caso contrario la soluzione è 0 (ossia la lunghezza della sequenza vuota). Pertanto, si può evitare di considerare tutte le coppie (i, j) con $i = 0 \vee j = 0$: il numero di sottoproblemi che compone il problema ausiliario P' possono essere ridotti a $m \cdot n$.

Riassumendo, il caso base si ha per quei

sottoproblemi di dimensione (i, j) con $i > 0 \wedge j > 0$ tali che $x_i \neq y_j$

ed è scrivibile come

$$c_{i,j} = 0$$

7.4.2 Passo ricorsivo: (i, j) con $i > 0$ e $j > 0$ tali che $x_i = y_j$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, j) tale che $x_i = y_j$, ossia quando i due prefissi X_i e Y_j considerati terminano con lo stesso elemento. In questo caso, la lunghezza della più lunga sottosequenza con colori alternanti comune fra X_i e Y_j è uguale alla lunghezza della più lunga

sottosequenza comune con colori alternanti calcolata per un sottoproblema di dimensione minore e che termina con un carattere con colore diverso da quello di $x_i = y_j$ aumentata di uno (in quanto si accoda l'elemento comune x_i alla sottosequenza comune relativa al sottoproblema considerato). Il tutto può essere scritto come:

$$c_{i,j} = 1 + \max\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, \phi(x_i) \neq \phi(x_h)\}$$

Poiché può accadere che l'insieme $\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, \phi(x_i) \neq \phi(x_h)\}$ sia vuoto (il che corrisponde al fatto che l'elemento $x_i = y_j$ è colorato allo stesso modo di tutti gli elementi precedenti e, quindi, non può essere accodato a nessuna più lunga sottosequenza alternante relativa a sottoproblemi di dimensione minore), assumiamo per definizione che $\max\{\emptyset\} = 0$, così che il corrispondente valore di $c_{i,j}$ risulti uguale a 1. Inoltre, si noti che l'insieme $\{c_{h,k} \mid 1 \leq h < i, 1 \leq k < j, x_h < x_i\}$ corrisponde all'insieme vuoto anche se $i = 1 \vee j = 1$ in quanto i casi con $i = 0 \vee j = 0$ sono stati omessi dal caso base e, pertanto, non esistono sottoproblemi di dimensione minore a quella del sottoproblema considerato (ossia quello con $i = 1 \vee j = 1$).

7.5 Soluzione del problema

Una volta calcolati i valori $c_{1,1}, c_{2,1}, c_{1,2}, \dots, c_{m,n}$ si hanno a disposizione tutte le lunghezze delle sottosequenze comuni massimali fra qualsiasi prefisso di X e qualsiasi prefisso di Y che non hanno elementi consecutivi dello stesso colore e che terminano con l'ultimo elemento di entrambi i prefissi. La soluzione al problema ausiliario P' è $c_{m,n}$ mentre quella del problema ridotto PBR è:

$$\max\{c_{i,j} \mid 1 \leq i \leq m \wedge 1 \leq j \leq n\}$$

Esempio Date $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$, tutti i valori $c_{i,j}$ relativi sono contenuti nella matrice presentata in Figura 7.

7.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 7.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j}$ di ogni sottoproblema. L'algoritmo 13 ne presenta lo pseudocodice: la prima chiamata deve essere $LACSRIC(m, n)$ (si suppone di aver accesso alle sequenze X e Y). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

7.7 Implementazione bottom-up

L'algoritmo 14 presenta lo pseudocodice che calcola tutti i valori $c_{0,0}, c_{0,1}, \dots, c_{m,n}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 7.4. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in

			1	2	3	4	5	6	7	8	j
			2	7	4	23	21	14	1	8	y_j
			r	b	r	g	g	g	r	b	$\varphi(y_j)$
1	2	r	1	0	0	0	0	0	0	0	
2	4	r	0	0	1	0	0	0	0	0	
3	7	b	0	2	0	0	0	0	0	0	
4	11	g	0	0	0	0	0	0	0	0	
5	21	g	0	0	0	0	3	0	0	0	
6	14	g	0	0	0	0	0	3	0	0	
7	1	r	0	0	0	0	0	0	4	0	
i	x_i	$\varphi(x_i)$									

Figura 7: Sottosequenza comune con colori alternanti: matrice di tutti i $c_{i,j}$ ottenuti considerando le sequenze $X = \langle 2, 4, 7, 11, 21, 14, 1 \rangle$ e $Y = \langle 2, 7, 4, 23, 21, 14, 1, 8 \rangle$. In rosso è evidenziata la soluzione del problema ridotto

Algorithm 13 Lunghezza di una più lunga sottosequenza con colori alternanti comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: ricorsione).

```

procedure LACSRIC( $i, j$ )
  if  $i = 0 \vee j = 0 \vee x_i \neq y_j$  then
    return 0
  else
     $max \leftarrow 0$ 
    for  $h \leftarrow 1$  to  $i - 1$  do
      for  $k \leftarrow 1$  to  $j - 1$  do
        if  $\phi(x_h) \neq \phi(x_i)$  then
           $S \leftarrow \text{LACSRIC}(h, k)$ 
          if  $S > max$  then
             $max \leftarrow S$ 
    return  $1 + max$ 

```

modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m^2 \cdot n^2)$ occupando $\Theta(m \cdot n)$ spazio in memoria (ossia una matrice che contiene i vari $c_{i,j}$).

Algorithm 14 Lunghezza di una più lunga sottosequenza con colori alternanti comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: bottom-up).

```

procedure LACS( $X, Y$ )
   $max \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      if  $x_i \neq y_j$  then
         $c[i, j] \leftarrow 0$ 
      else
         $temp \leftarrow 0$ 
        for  $h \leftarrow 1$  to  $i - 1$  do
          for  $k \leftarrow 1$  to  $j - 1$  do
            if  $(\phi(x_h) \neq \phi(x_i)) \wedge (c[h, k] > temp)$  then
               $temp \leftarrow c[h, k]$ 
           $c[i, j] \leftarrow 1 + temp$ 
        if  $c[i, j] > max$  then
           $max \leftarrow c[i, j]$ 
  return  $max$ 

```

8 Più lunga sottosequenza comune di lunghezza almeno L (LCS $\geq L$)

8.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m e n numeri interi, e un naturale L , stabilire se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X e Y è maggiore o uguale a L

Ad esempio, date $X = \langle 2, 4, 7, 11, 14, 21, 8 \rangle$, $Y = \langle 2, 7, 4, 23, 11, 14, 21, 1 \rangle$ e $L = 4$, la soluzione al problema deve essere *true* in quanto tutte le più lunghe sottosequenze comuni hanno lunghezza 5 (che è maggiore di 4).

Per risolvere il problema PB non è necessario introdurre alcuna versione ridotta.

8.2 Sottoproblemi e variabili associate

Il problema PB contiene diversi sottoproblemi ognuno dei quali non ha come input la tripla (X, Y, L) ma una tripla composta dai prefissi delle sequenze X e Y e da un certo naturale $0 \leq l \leq L$.

Il **sottoproblema di dimensione** (i, j, l) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, e un naturale l , stabilire se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è maggiore o uguale a l

Dato che $0 \leq i \leq m$, $0 \leq j \leq n$ e $0 \leq l \leq L$, si ottengono $(m+1) \cdot (n+1) \cdot (L+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j, l) , la variabile ad esso associata è $c_{i,j,l}$ ed è così definita:

$c_{i,j,l} = \mathbf{true}$ se e solo se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è maggiore o uguale a l , **false** altrimenti

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j, l) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(1, 1, 1)$, il valore della variabile $c_{1,1,1}$ si otterrà utilizzando le variabili $c_{0,0,0}$, $c_{0,0,1}$, $c_{1,0,0}$, $c_{1,0,1}$, $c_{0,1,0}$ e $c_{0,1,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j, l) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare tutte le variabili $c_{i',j',l'}$ relative ai sottoproblemi di dimensione

minore senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j,l}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j, l) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) e utilizzando il valore l ?

8.3 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

8.3.1 Caso base

Il caso base si ha in due situazioni differenti:

1. per un qualunque sottoproblema di dimensione (i, j, l) con

$$l = 0$$

2. per un qualunque sottoproblema di dimensione (i, j, l) con

$$l > 0 \wedge (i < l \vee j < l)$$

Per entrambi questi casi, infatti, è facile stabilire il valore di $c_{i,j,l}$.

Caso base 1: (i, j, l) con $l = 0$ Questo caso base si verifica quando si richiede che una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j abbia lunghezza maggiore o uguale a 0. È facile ottenere il valore della variabile $c_{i,j,0}$ in quanto vale sempre *true*: c'è sempre una sottosequenza comune di lunghezza maggiore o uguale a 0. Infatti, nel caso migliore si ha una sottosequenza comune di lunghezza maggiore a 0 mentre nel caso peggiore, si ha una sottosequenza comunque di lunghezza 0 (ossia la lunghezza di una sottosequenza comune fra la sequenza vuota e una qualsiasi altra sequenza). Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = \text{true} \quad \text{se } l = 0$$

Caso base 2: (i, j, l) con $l > 0 \wedge (i < l \vee j < l)$ Questo caso base si verifica quando uno (o entrambi) i prefissi considerati hanno lunghezza minore di l e si richiede che una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j abbia lunghezza maggiore a 0. In questo caso, è facile ottenere il valore della variabile $c_{i,j,0}$ in quanto vale sempre *false*: dato che il problema iniziale richiede che la più lunga sottosequenza comune sia lunga almeno l , è impossibile ottenerla se si considerano prefissi più corti di l . Si noti che questo caso base include anche il caso nel quale si richiede che $i = 0$ o $j = 0$ e $l > 0$: anche in questo caso, il valore di $c_{i,j,l}$ deve essere *false*. Oltre alla spiegazione data precedentemente, in questo caso si può anche considerare che la lunghezza di una qualunque tra le più lunghe sottosequenze comuni fra la sequenza vuota e una qualsiasi altra sequenza è pari a 0 (ossia la lunghezza della sequenza vuota). Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = \text{false} \quad \text{se } l > 0 \wedge (i < l \vee j < l)$$

Riassumendo, il caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = \begin{cases} \text{true} & \text{se } l = 0 \\ \text{false} & \text{se } l > 0 \wedge (i < l \vee j < l) \end{cases}$$

8.3.2 Passo ricorsivo

Il passo ricorsivo si ha nelle seguenti situazioni:

1. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i = y_j$$

2. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i \neq y_j$$

Passo ricorsivo 1: (i, j, l) con $i \geq l \wedge j \geq l \wedge l > 0$ **tale che** $x_i = y_j$ Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi di lunghezza $\geq l$ e che terminano con lo stesso elemento.

Se i due prefissi terminano con lo stesso elemento ($x_i = y_j$) allora la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è maggiore o uguale a l se e solo se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} risulta essere maggiore o uguale a $l-1$ (si considera la soluzione $c_{i-1,j-1,l-1}$ relativa al sottoproblema di dimensione $(i-1, j-1, l-1)$ con input il prefisso X_{i-1} , il prefisso Y_{j-1} e la lunghezza $l-1$). Infatti, solo se esiste una più lunga sottosequenze comune a X_{i-1} e Y_{j-1} di lunghezza maggiore o uguale a $l-1$ allora è possibile concatenare ad essa l'elemento comune $x_i = y_j$ ed ottenere una sottosequenza comune a X_i e Y_j di lunghezza maggiore o uguale a l .

Riassumendo, questo passo ricorsivo può essere scritto come:

$$c_{i,j,l} = c_{i-1,j-1,l-1} \quad \text{se } i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i = y_j$$

Passo ricorsivo 2: (i, j, l) con $i \geq l \wedge j \geq l \wedge l > 0$ **tale che** $x_i \neq y_j$
Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi di lunghezza $\geq l$ e che terminano con due elementi diversi.

Se i due prefissi terminano con due elementi diversi ($x_i \neq y_j$) allora la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è maggiore o uguale a l se e solo se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_{i-1} e Y_j risulta essere maggiore o uguale a l (si considera la soluzione $c_{i-1,j,l}$ relativa al sottoproblema di dimensione $(i-1, j, l)$ con input il prefisso X_{i-1} , il prefisso Y_j e la lunghezza l : si è tolto da X_i l'elemento x_i) oppure la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_{j-1} risulta essere maggiore o uguale a l (si considera la soluzione $c_{i,j-1,l}$ relativa al sottoproblema di dimensione $(i, j-1, l)$ con input il prefisso X_i , il prefisso Y_{j-1} e la lunghezza l : si è tolto da Y_j l'elemento y_j). Infatti, dato che gli elementi terminanti dei due prefissi sono diversi e non possono essere accodati a nessuna soluzione di un sottoproblema di dimensione minore, solo se esiste una più lunga sottosequenza comune a X_{i-1} e Y_j di lunghezza maggiore o uguale a l oppure una più lunga sottosequenza comune a X_i e Y_{j-1} di lunghezza maggiore o uguale a l è possibile affermare che una più lunga sottosequenza comune a X_i e Y_j ha lunghezza maggiore o uguale a l . Siccome non possiamo sapere a priori quale dei due casi si verifichi, consideriamo la soluzione di entrambi i sottoproblemi di dimensione $(i-1, j, l)$ e $(i, j-1, l)$ e verifichiamo che almeno una delle due valga *true*. Formalmente, possiamo scrivere:

$$c_{i,j,l} = c_{i-1,j,l} \vee c_{i,j-1,l} \quad \text{se } i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i \neq y_j$$

Riassumendo, il passo ricorsivo può essere scritto come:

$$c_{i,j,l} = \begin{cases} c_{i-1,j-1,l-1} & \text{se } i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i = y_j \\ c_{i-1,j,l} \vee c_{i,j-1,l} & \text{se } i \geq l \wedge j \geq l \wedge l > 0 \wedge x_i \neq y_j \end{cases}$$

8.4 Soluzione del problema

Una volta calcolati i valori di tutte le variabili $c_{0,0,0}, \dots, c_{m,n,L}$ è possibile identificare la soluzione al problema PB:

$$c_{m,n,L}$$

8.5 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 8.3, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j,l}$ di ogni sottoproblema. L'algoritmo 15 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LCS} \geq \text{LRIC}(m, n, L)$ (si suppone di aver accesso alle sequenze X e Y e al valore L). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta.

In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 15 Esistenza di una più lunga sottosequenza di lunghezza $\geq L$ comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: ricorsione).

```

procedure LCS $\geq$ LRIC( $i, j, l$ )
  if  $l = 0$  then
    return true
  else if  $l > 0 \wedge (i < l \vee j < l)$  then
    return false
  else
    if  $x_i = y_j$  then
      return LCS $\geq$ LRIC( $i - 1, j - 1, l - 1$ )
    else
      return LCS $\geq$ LRIC( $i - 1, j, l$ )  $\vee$  LCS $\geq$ LRIC( $i, j - 1, l$ )

```

8.6 Implementazione bottom-up

L'algoritmo 16 presenta lo pseudocodice che calcola tutti i valori $c_{0,0,0}, \dots, c_{m,n,L}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 8.3. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m \cdot n \cdot L)$ occupando $\Theta(m \cdot n \cdot L)$ spazio in memoria (ossia una matrice tridimensionale che contiene i vari $c_{i,j,L}$).

Algorithm 16 Esistenza di una più lunga sottosequenza di lunghezza $\geq L$ comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: bottom-up).

```

procedure LCS $\geq$ L( $X, Y, L$ )
  for  $i \leftarrow 0$  to  $m$  do
    for  $j \leftarrow 0$  to  $n$  do
       $c[i, j, 0] \leftarrow \text{true}$ 
  for  $l \leftarrow 1$  to  $L$  do
    for  $i \leftarrow 0$  to  $m$  do
      for  $j \leftarrow 0$  to  $\text{MIN}(n, l - 1)$  do
         $c[i, j, l] \leftarrow \text{false}$ 
  for  $l \leftarrow 1$  to  $L$  do
    for  $i \leftarrow 0$  to  $\text{MIN}(m, l - 1)$  do
      for  $j \leftarrow l$  to  $n$  do
         $c[i, j, l] \leftarrow \text{false}$ 
  for  $l \leftarrow 1$  to  $L$  do
    for  $i \leftarrow l$  to  $m$  do
      for  $j \leftarrow l$  to  $n$  do
        if  $x_i = y_j$  then
           $c[i, j, l] \leftarrow c[i - 1, j - 1, l - 1]$ 
        else
           $c[i, j, l] \leftarrow c[i - 1, j, l] \vee c[i, j - 1, l]$ 
  return  $c[m, n, L]$ 

```

9 Più lunga sottosequenza comune di lunghezza al più L ($\text{LCS} \leq L$)

9.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m e n numeri interi, e un naturale L , stabilire se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X e Y è minore o uguale a L

Ad esempio, date $X = \langle 2, 4, 7, 11, 14, 21, 8 \rangle$, $Y = \langle 2, 7, 4, 23, 11, 14, 21, 1 \rangle$ e $L = 4$, la soluzione al problema deve essere *false* in quanto tutte le più lunghe sottosequenze comuni hanno lunghezza 5 (che è maggiore di 4).

Per risolvere il problema PB non è necessario introdurre alcuna versione ridotta.

9.2 Sottoproblemi e variabili associate

Il problema PB contiene diversi sottoproblemi ognuno dei quali non ha come input la tripla (X, Y, L) ma una tripla composta dai prefissi delle sequenze X e Y e da un certo naturale $0 \leq l \leq L$.

Il **sottoproblema di dimensione** (i, j, l) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, e un naturale l , stabilire se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è minore o uguale a l

Dato che $0 \leq i \leq m$, $0 \leq j \leq n$ e $0 \leq l \leq L$, si ottengono $(m+1) \cdot (n+1) \cdot (L+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j, l) , la variabile ad esso associata è $c_{i,j,l}$ ed è così definita:

$c_{i,j,l} = \text{true}$ se e solo se la lunghezza di una qualunque tra le più lunghe sottosequenze comuni a X_i e Y_j è minore o uguale a l , **false** altrimenti

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j, l) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(1, 1, 1)$, il valore della variabile $c_{1,1,1}$ si otterrà utilizzando le variabili $c_{0,0,0}$, $c_{0,0,1}$, $c_{1,0,0}$, $c_{1,0,1}$, $c_{0,1,0}$ e $c_{0,1,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j, l) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare tutte le variabili $c_{i',j',l'}$ relative ai sottoproblemi di dimensione

minore senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j,l}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j, l) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) e utilizzando il valore l ?

9.3 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

9.3.1 Caso base

Il caso base si ha in due situazioni differenti:

1. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i = 0 \vee j = 0$$

2. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i > 0 \wedge j > 0 \wedge l = 0 \text{ tale che } x_i = y_j$$

Per entrambi questi casi, infatti, è facile stabilire il valore di $c_{i,j,l}$.

Caso base 1: (i, j, l) con $i = 0 \vee j = 0$ Questo caso base si verifica quando uno dei due prefissi considerati è la sequenza vuota. È facile ottenere il valore della variabile $c_{i,j,l}$: per qualsiasi valore di l , $c_{i,j,l}$ vale *true* in quanto la lunghezza della più lunga sottosequenza comune fra la sequenza vuota e una qualsiasi altra sequenza è pari a 0 (che è minore o uguale di qualsiasi valore può assumere l). Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = \text{true se } i = 0 \vee j = 0$$

Caso base 2: (i, j, l) con $i > 0 \wedge j > 0 \wedge l = 0$ **tale che** $x_i = y_j$

Si capirà che questo è un caso base una volta affrontato il passo ricorsivo 1 (Sezione 9.3.2)

Questo caso base si verifica quando entrambi i prefissi considerati sono diversi dalla stringa vuota, il loro ultimo elemento è uguale e si richiede che tutte le più lunghe sottosequenze comuni abbiano lunghezza minore o uguale a 0. Questo è sempre falso in quanto $x_i = y_j$ e una fra le più lunghe sottosequenze comuni a X_i e Y_j deve contenere per forza l'elemento $x_i = y_j$ e quindi deve avere lunghezza almeno 1 (che non è minore o uguale a $l = 0$). Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = false \quad se \ i > 0 \wedge j > 0 \wedge l = 0 \wedge x_i = y_j$$

Riassumendo, il caso base può essere scritto nel seguente modo:

$$c_{i,j,l} = \begin{cases} true & se \ i = 0 \vee j = 0 \\ false & se \ i > 0 \wedge j > 0 \wedge l = 0 \wedge x_i = y_j \end{cases}$$

9.3.2 Passo ricorsivo

Il passo ricorsivo si ha nelle seguenti situazioni:

1. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i > 0 \wedge j > 0 \wedge l > 0 \wedge x_i = y_j$$

2. per un qualunque sottoproblema di dimensione (i, j, l) con

$$i > 0 \wedge j > 0 \wedge l \geq 0 \wedge x_i \neq y_j$$

Passo ricorsivo 1: (i, j, l) con $i > 0 \wedge j > 0 \wedge l > 0$ **tale che** $x_i = y_j$
Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota che terminano con lo stesso elemento.

Se i due prefissi terminano con lo stesso elemento ($x_i = y_j$) allora la lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j è minore o uguale a l se e solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} hanno lunghezza minore o uguale a $l-1$ (si considera la soluzione $c_{i-1,j-1,l-1}$ relativa al sottoproblema di dimensione $(i-1, j-1, l-1)$ con input il prefisso X_{i-1} , il prefisso Y_{j-1} e la lunghezza $l-1$). Infatti, solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} hanno lunghezza minore o uguale a $l-1$ allora è possibile concatenare ad ognuna di esse l'elemento comune $x_i = y_j$ ed ottenere una qualsiasi sottosequenza comune a X_i e Y_j di lunghezza minore o uguale a l . Tutto questo, però, è corretto solamente se $l \geq 0$ in quanto se l fosse uguale a 0, si dovrebbe andare a considerare la soluzione di un sottoproblema di dimensione $(i-1, j-1, -1)$ che non esiste.

Riassumendo, questo passo ricorsivo può essere scritto come:

$$c_{i,j,l} = c_{i-1,j-1,l-1} \quad se \ i > 0 \wedge j > 0 \wedge l > 0 \wedge x_i = y_j$$

Nel caso in cui $l = 0$, il sottoproblema è facile da risolvere senza dover ricorrere all'utilizzo delle soluzioni dei sottoproblemi di dimensione minore: quando $l = 0$ il caso qui descritto è un caso base (si veda il caso base 2 nella Sezione 9.3.1).

Passo ricorsivo 2: (i, j, l) con $i > 0 \wedge j > 0 \wedge l \geq 0$ **tale che** $x_i \neq y_j$
Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota che terminano con due elementi diversi. In questo caso, la lunghezza l non è significativa: il valore della soluzione $c_{i,j,l}$ è lo stesso per qualsiasi valore di $l \geq 0$.

Se i due prefissi terminano con due elementi diversi ($x_i \neq y_j$) allora la lunghezza di una tra le più lunghe sottosequenze comuni a X_i e Y_j è minore o uguale a l se e solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_j hanno lunghezza minore o uguale a l (si considera la soluzione $c_{i-1,j,l}$, ossia la soluzione del sottoproblema di dimensione $(i-1, j, l)$ con input il prefisso X_{i-1} , il prefisso Y_j e la lunghezza l : si è tolto da X_i l'elemento x_i) E tutte le più lunghe sottosequenze comuni a X_i e Y_{j-1} hanno lunghezza minore o uguale a l (si considera la soluzione $c_{i,j-1,l}$, ossia la soluzione del sottoproblema di dimensione $(i, j-1, l)$ con input il prefisso X_i , il prefisso Y_{j-1} e la lunghezza l : si è tolto da Y_j l'elemento y_j). Infatti, dato che gli elementi terminanti dei due prefissi sono diversi e non possono essere accodati a nessuna soluzione di un sottoproblema di dimensione minore, solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_j e tutte le più lunghe sottosequenze comuni a X_i e Y_{j-1} hanno lunghezza minore o uguale a l allora è possibile affermare che una qualsiasi fra le più lunghe sottosequenze comuni a X_i e Y_j ha lunghezza minore o uguale a l . Siccome non possiamo sapere a priori quale dei due casi si verifichi, consideriamo la soluzione di entrambi i sottoproblemi di dimensione $(i-1, j, l)$ e $(i, j-1, l)$ e verifichiamo che entrambe valgano *true*. Formalmente, possiamo scrivere:

$$c_{i,j,l} = c_{i-1,j,l} \wedge c_{i,j-1,l} \quad \text{se } i > 0 \wedge j > 0 \wedge l \geq 0 \wedge x_i \neq y_j$$

Riassumendo, il passo ricorsivo può essere scritto come:

$$c_{i,j,l} = \begin{cases} c_{i-1,j-1,l-1} & \text{se } i > 0 \wedge j > 0 \wedge l > 0 \wedge x_i = y_j \\ c_{i-1,j,l} \wedge c_{i,j-1,l} & \text{se } i > 0 \wedge j > 0 \wedge l \geq 0 \wedge x_i \neq y_j \end{cases}$$

9.4 Soluzione del problema

Una volta calcolati i valori di tutte le variabili $c_{0,0,0}, \dots, c_{m,n,L}$ è possibile identificare la soluzione al problema PB:

$$c_{m,n,L}$$

9.5 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 9.3, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j,l}$ di ogni sottoproblema. L'algoritmo 17 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LCS} \leq \text{LRIC}(m, n, L)$ (si suppone di aver accesso alle sequenze X e Y). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette

di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 17 Esistenza di una più lunga sottosequenza di lunghezza $\leq L$ comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: ricorsione).

```

procedure LCS $\leq$ LRIC( $i, j, l$ )
  if  $i = 0 \vee j = 0$  then
    return true
  else if  $i > 0 \wedge j = 0 \wedge l = 0 \wedge x_i = y_j$  then
    return false
  else
    if  $x_i = y_j$  then
      return LCS $\leq$ LRIC( $i - 1, j - 1, l - 1$ )
    else
      return LCS $\leq$ LRIC( $i - 1, j, l$ )  $\wedge$  LCS $\leq$ L( $i, j - 1, l$ )

```

9.6 Implementazione bottom-up

L'algoritmo 18 presenta lo pseudocodice che calcola tutti i valori $c_{0,0,0}, \dots, c_{m,n,L}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 9.3. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m \cdot n \cdot L)$ occupando $\Theta(m \cdot n \cdot L)$ spazio in memoria (ossia una matrice tridimensionale che contiene i vari $c_{i,j,L}$).

Algorithm 18 Esistenza di una più lunga sottosequenza di lunghezza $\leq L$ comune fra due sequenze X e Y di lunghezza m e n rispettivamente (tecnica: bottom-up).

```

procedure LCS $\leq$ L( $X, Y, L$ )
  for  $i \leftarrow 0$  to  $m$  do
    for  $l \leftarrow 0$  to  $L$  do
       $c[i, 0, l] \leftarrow \text{true}$ 
  for  $j \leftarrow 0$  to  $n$  do
    for  $l \leftarrow 0$  to  $L$  do
       $c[0, j, l] \leftarrow \text{true}$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      if  $x_i = y_j$  then
         $c[i, j, 0] \leftarrow \text{false}$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      for  $l \leftarrow 0$  to  $L$  do
        if  $x_i = y_j$  then
          if  $l > 0$  then
             $c[i, j, l] \leftarrow c[i - 1, j - 1, l - 1]$ 
          else
             $c[i, j, l] \leftarrow c[i - 1, j, l] \wedge c[i, j - 1, l]$ 
  return  $c[m, n, L]$ 

```

10 Più lunga sottosequenza comune con al più R elementi rossi (LCSR)

Premessa Sia C un insieme di colori. Gli elementi della sequenza considerata in questo esercizio sono numeri naturali colorati. La funzione di colorazione è definita nel seguente modo: $\phi : \mathbb{N} \rightarrow C$. Per esempio, data la funzione:

$$\phi(x) = \begin{cases} \text{rosso} & x < 5 \\ \text{blu} & 5 \leq x \leq 10 \\ \text{verde} & x > 10 \end{cases}$$

la sequenza $X = \langle 2, 4, 7, 6, 11, 3, 21, 14, 1 \rangle$ sarà colorata nel seguente modo: $\langle r, r, b, b, g, r, g, g, r \rangle$.

10.1 Definizione del problema

Il problema PB è definito come segue:

date due sequenze X e Y , rispettivamente di m e n numeri interi, e un naturale R , stabilire se tutte le più lunghe sottosequenze comuni a X e Y contengono al più R elementi colorati di rosso

Ad esempio, dati $X = \langle 2, 7, 3, 11, 21, 14, 1 \rangle$, $Y = \langle 2, 3, 7, 23, 21, 14, 1, 8 \rangle$ e $R = 4$, la soluzione al problema deve essere *true* in quanto tutte le più lunghe sottosequenze comuni ($\langle 2, 7, 21, 14, 1 \rangle$ e $\langle 2, 3, 21, 14, 1 \rangle$) contengono, rispettivamente, 2 e 3 elementi colorati di rosso.

Per risolvere il problema PB non è necessario introdurre alcuna versione ridotta.

10.2 Sottoproblemi e variabili associate

Il problema PB contiene diversi sottoproblemi ognuno dei quali non ha come input la tripla (X, Y, R) ma una tripla composta dai prefissi delle sequenze X e Y e da un certo naturale $0 \leq r \leq R$.

Il **sottoproblema di dimensione** (i, j, r) è definito come segue:

date due sequenze X e Y , rispettivamente di m ed n numeri interi, e un naturale r , stabilire se tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso

Dato che $0 \leq i \leq m$, $0 \leq j \leq n$ e $0 \leq r \leq R$, si ottengono $(m+1) \cdot (n+1) \cdot (R+1)$ sottoproblemi (i e j possono valere 0 in quanto si deve considerare anche il caso in cui un prefisso sia la sequenza vuota). Ad ogni sottoproblema di PBR è associata una **variabile**.

Considerato il sottoproblema di dimensione (i, j, r) , la variabile ad esso associata è $c_{i,j,r}$ ed è così definita:

$c_{i,j,r} = \text{true}$ se e solo se tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso, **false** altrimenti

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione (i, j, r) , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(1, 1, 1)$, il valore della variabile $c_{1,1,1}$ si otterrà utilizzando le variabili $c_{0,0,0}$, $c_{0,0,1}$, $c_{1,0,0}$, $c_{1,0,1}$, $c_{0,1,0}$ e $c_{0,1,1}$. Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione (i, j, r) , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare tutte le variabili $c_{i',j',r'}$ relative ai sottoproblemi di dimensione minore senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione $c_{i,j,r}$ andando a combinare le soluzioni dei sottoproblemi di dimensione minore a (i, j, r) ed, eventualmente, accodando gli elementi x_i e y_j estratti dall'input (X_i, Y_j) e utilizzando il valore r ?

10.3 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

10.3.1 Caso base

Il caso base si ha in due situazioni differenti:

1. per un qualunque sottoproblema di dimensione (i, j, r) con

$$i = 0 \vee j = 0$$

2. per un qualunque sottoproblema di dimensione (i, j, r) con

$$i > 0 \wedge j > 0 \wedge r = 0 \text{ tale che } x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$$

Per entrambi questi casi, infatti, è facile stabilire il valore di $c_{i,j,r}$.

Caso base 1: (i, j, r) con $i = 0 \vee j = 0$ Questo caso base si verifica quando uno dei due prefissi considerati è la sequenza vuota. È facile ottenere il valore della variabile $c_{i,j,r}$: dato che si richiede che tutte le più lunghe sottosequenze comuni contengano al più r elementi colorati di rosso, l'unica più lunga sottosequenza comune fra la sequenza vuota e una qualsiasi sequenza è la sequenza vuota stessa che non contiene alcun elemento e quindi, dato che contiene 0 elementi colorati di rossi, certamente contiene anche al più r elementi colorati di rosso (qualsiasi sia r). Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,r} = \text{true} \quad \text{se } i = 0 \vee j = 0$$

Caso base 2: (i, j, r) con $i > 0 \wedge j > 0 \wedge r = 0$ **tale che** $x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$

Si capirà che questo è un caso base una volta affrontato il passo ricorsivo 1 (Sezione 10.3.2)

Questo caso base si verifica quando entrambi i prefissi considerati sono diversi dalla stringa vuota, il loro ultimo elemento è uguale e colorato di rosso ma si richiede che tutte le più lunghe sottosequenze comuni contengano al più 0 elementi colorati di rosso. Questo è impossibile in quanto $x_i = y_j$ e almeno una più lunga sottosequenza comune a X_i e Y_j deve contenere per forza questo elemento che è colorato di rosso. Pertanto, non tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono almeno 1 elemento colorato di rosso: il valore della variabile $c_{i,j,r}$ deve essere false. Questo caso base può essere scritto nel seguente modo:

$$c_{i,j,r} = \text{false} \quad \text{se } i > 0 \wedge j > 0 \wedge r = 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$$

Riassumendo, il caso base può essere scritto nel seguente modo:

$$c_{i,j,r} = \begin{cases} \text{true} & \text{se } i = 0 \vee j = 0 \\ \text{false} & \text{se } i > 0 \wedge j > 0 \wedge r = 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso} \end{cases}$$

10.3.2 Passo ricorsivo

Il passo ricorsivo si ha nelle seguenti situazioni:

1. per un qualunque sottoproblema di dimensione (i, j, r) con

$$i > 0 \wedge j > 0 \wedge r > 0 \text{ tale che } x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$$

2. per un qualunque sottoproblema di dimensione (i, j, r) con

$$i > 0 \wedge j > 0 \wedge r \geq 0 \text{ tale che } x_i = y_j \wedge \phi(x_i) = \phi(y_j) \neq \text{rosso}$$

3. per un qualunque sottoproblema di dimensione (i, j, r) con

$$i > 0 \wedge j > 0 \wedge r \geq 0 \text{ tale che } x_i \neq y_j$$

Passo ricorsivo 1: (i, j, r) con $i > 0 \wedge j > 0$ *tale che* $x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$ Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota che terminano con lo stesso elemento colorato di rosso.

Se i due prefissi terminano con lo stesso elemento ($x_i = y_j$) e questo è colorato di rosso allora tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso se e solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} contengono al più $r - 1$ elementi colorati di rosso (si considera la soluzione $c_{i-1, j-1, r-1}$ relativa al sottoproblema di dimensione $(i-1, j-1, r-1)$ con input il prefisso X_{i-1} , il prefisso Y_{j-1} e $r-1$). Infatti, solo se tutte le sottosequenze più lunghe comuni a X_{i-1} e Y_{j-1} contengono al più $r - 1$ elementi colorati di rosso allora è possibile concatenare ad esse l'elemento comune $x_i = y_j$ ed ottenere una qualsiasi più lunga sottosequenza comune a X_i e Y_j con al più r elementi colorati di rosso. Tutto questo, però, è corretto solamente se $r > 0$ in quanto se r fosse uguale a 0, si dovrebbe andare a considerare la soluzione di un sottoproblema di dimensione $(i-1, j-1, -1)$ che non esiste.

Riassumendo, questo passo ricorsivo può essere scritto come:

$$c_{i,j,r} = c_{i-1,j-1,r-1} \text{ se } i > 0 \wedge j > 0 \wedge r > 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso}$$

Nel caso in cui $r = 0$, il sottoproblema è facile da risolvere senza dover ricorrere all'utilizzo delle soluzioni dei sottoproblemi di dimensione minore: quando $r = 0$ il caso qui descritto è un caso base (si veda il caso base 2 nella Sezione 10.3.1).

Passo ricorsivo 2: (i, j, r) con $i > 0 \wedge j > 0 \wedge r \geq 0$ *tale che* $x_i = y_j \wedge \phi(x_i) = \phi(y_j) \neq \text{rosso}$ Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi $X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota che terminano con lo stesso elemento colorato non di rosso. In questo caso, il valore r non è significativo (è necessario che $r \geq 0$): il valore della soluzione $c_{i,j,r}$ è lo stesso per qualsiasi valore di r .

Se i due prefissi terminano con lo stesso elemento che non è però colorato di rosso allora tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso se e solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} contengono al più r elementi colorati di rosso (si considera la soluzione $c_{i-1, j-1, r}$, ossia la soluzione del sottoproblema di dimensione $(i-1, j-1, r)$ con input il prefisso X_{i-1} , il prefisso Y_{j-1} e il valore r). Infatti, solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_{j-1} contengono al più r elementi colorati di rosso allora è possibile concatenare a qualunque sottosequenza di esse l'elemento comune $x_i = y_j$ e poter affermare ancora che tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso (si ricorda che l'elemento da concatenare non è colorato di rosso). Il tutto può essere scritto come:

$$c_{i,j,r} = c_{i-1,j-1,r} \text{ se } i > 0 \wedge j > 0 \wedge r \geq 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) \neq \text{rosso}$$

Passo ricorsivo 3: (i, j, r) con $i > 0 \wedge j > 0 \wedge r \geq 0$ *tale che* $x_i \neq y_j$ Questo passo ricorsivo si verifica quando si vanno a considerare due prefissi

$X_i = \langle x_0, x_1, \dots, x_i \rangle$ e $Y_j = \langle y_0, y_1, \dots, y_j \rangle$ entrambi diversi dalla sequenza vuota che terminano con due elementi differenti. In questo caso, il colore degli elementi e il valore r non sono significativi (è necessario che $r \geq 0$): il valore della soluzione $c_{i,j,r}$ è lo stesso per qualsiasi colore e per qualsiasi valore di r .

Se i due prefissi terminano con due elementi differenti, tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso se e solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_j contengono al più r elementi colorati di rosso (si considera la soluzione $c_{i-1,j,r}$, ossia la soluzione del sottoproblema di dimensione $(i-1, j, r)$ con input il prefisso X_{i-1} , il prefisso Y_j e il valore r : si è tolto da X_i l'elemento x_i) e tutte le più lunghe sottosequenze comuni a X_i e Y_{j-1} contengono al più r elementi colorati di rosso (si considera la soluzione $c_{i,j-1,r}$, ossia la soluzione del sottoproblema di dimensione $(i, j-1, r)$ con input il prefisso X_i , il prefisso Y_{j-1} e il valore r : si è tolto da Y_j l'elemento y_j). Infatti, solo se tutte le più lunghe sottosequenze comuni a X_{i-1} e Y_j contengono al più r elementi colorati di rosso e tutte le più lunghe sottosequenze comuni a X_i e Y_{j-1} contengono al più r elementi colorati di rosso allora tutte le più lunghe sottosequenze comuni a X_i e Y_j contengono al più r elementi colorati di rosso. Siccome non possiamo sapere a priori quale dei due casi si verifichi, consideriamo la soluzione di entrambi i sottoproblemi di dimensione $(i-1, j, r)$ e $(i, j-1, r)$ e verifichiamo che entrambe valgano *true*. Formalmente, possiamo scrivere:

$$c_{i,j,r} = c_{i-1,j,r} \wedge c_{i,j-1,r} \quad \text{se } i > 0 \wedge j > 0 \wedge r \geq 0 \wedge x_i \neq y_j$$

Riassumendo, il passo ricorsivo può essere scritto come:

$$c_{i,j,r} = \begin{cases} c_{i-1,j-1,r-1} & \text{se } i > 0 \wedge j > 0 \wedge r > 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) = \text{rosso} \\ c_{i-1,j-1,r} & \text{se } i > 0 \wedge j > 0 \wedge r \geq 0 \wedge x_i = y_j \wedge \phi(x_i) = \phi(y_j) \neq \text{rosso} \\ c_{i-1,j,r} \wedge c_{i,j-1,r} & \text{se } i > 0 \wedge j > 0 \wedge r \geq 0 \wedge x_i \neq y_j \end{cases}$$

10.4 Soluzione del problema

Una volta calcolati i valori di tutte le variabili $c_{0,0,0}, \dots, c_{m,n,R}$ è possibile identificare la soluzione al problema PB:

$$c_{m,n,R}$$

10.5 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 10.3, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione $c_{i,j,r}$ di ogni sottoproblema. L'algoritmo 19 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{LCSRIC}(m, n, R)$ (si suppone di aver accesso alle sequenze X e Y). In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo

modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 19 Esistenza di una più lunga sottosequenza comune fra due sequenze X e Y di lunghezza m e n rispettivamente con al più R elementi colorati di rosso (tecnica: ricorsione).

```

procedure LCSRRIC( $i, j, r$ )
  if  $i = 0 \vee j = 0$  then
    return true
  else if  $i > 0 \wedge j > 0 \wedge r = 0 \wedge x_i = y_j \wedge \phi(x_i) = \text{rosso}$  then
    return false
  if  $x_i = y_j$  then
    if  $\phi(x_i) = \text{rosso}$  then
      return LCSRRIC( $i - 1, j - 1, r - 1$ )
    else
      return LCSRRIC( $i - 1, j - 1, r$ )
  else
    return LCSRRIC( $i - 1, j, r$ )  $\wedge$  LCSRRIC( $i, j - 1, r$ )

```

10.6 Implementazione bottom-up

L'algoritmo 20 presenta lo pseudocodice che calcola tutti i valori $c_{0,0,0}, \dots, c_{m,n,R}$ basandosi sulle equazioni di ricorrenza definite nella Sezione 10.3. Aniché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(m \cdot n \cdot R)$ occupando $\Theta(m \cdot n \cdot R)$ spazio in memoria (ossia una matrice tridimensionale che contiene i vari $c_{i,j,R}$).

Algorithm 20 Esistenza di una più lunga sottosequenza comune fra due sequenze X e Y di lunghezza m e n rispettivamente con al più R elementi colorati di rosso (tecnica: bottom-up).

```

procedure LCSR( $X, Y, R$ )
  for  $i \leftarrow 0$  to  $m$  do
    for  $r \leftarrow 0$  to  $R$  do
       $c[i, 0, r] \leftarrow \text{true}$ 
  for  $j \leftarrow 0$  to  $n$  do
    for  $r \leftarrow 0$  to  $R$  do
       $c[0, j, r] \leftarrow \text{true}$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      if  $x_i = y_j$  and  $\phi(x_i) = \text{rosso}$  then
         $c[i, j, 0] \leftarrow \text{false}$ 
  for  $i \leftarrow 1$  to  $m$  do
    for  $j \leftarrow 1$  to  $n$  do
      for  $r \leftarrow 0$  to  $R$  do
        if  $x_i = y_j$  then
          if  $\phi(x_i) = \text{rosso}$  then
            if  $r > 0$  then
               $c[i, j, r] \leftarrow c[i - 1, j - 1, r - 1]$ 
          else
             $c[i, j, r] \leftarrow c[i - 1, j - 1, r]$ 
        else
           $c[i, j, r] \leftarrow c[i - 1, j, r] \wedge c[i, j - 1, r]$ 
  return  $c[m, n, R]$ 

```

11 Weighted Interval Scheduling

11.1 Definizioni preliminari

Sia $A = \{1, 2, \dots, n\}$ un insieme di n attività, con $n \in \mathbb{N}$. Ad ogni attività $i \in A$ sono associate le seguenti informazioni $([s_i, e_i), v_i)$, dove:

- s_i indica il tempo di inizio dell'attività
- e_i , con $e_i \geq s_i$, indica il tempo di fine dell'attività (si noti che e_i non appartiene all'intervallo che specifica la durata dell'attività)
- v_i indica il valore dell'attività

Due attività $i, j \in A$ sono dette *compatibili* se non si sovrappongono:

$$[s_i, e_i) \cap [s_j, e_j) = \emptyset$$

Un insieme A di attività *contiene attività mutualmente compatibili* se e solo se per tutte le coppie (i, j) con $i \neq j$, l'attività i è compatibile con l'attività j :

$$\forall i, j \in A \text{ con } i \neq j, [s_i, e_i) \cap [s_j, e_j) = \emptyset$$

Definiamo ora la funzione

$$\text{COMP} : \mathcal{P}(\{1, \dots, n\}) \rightarrow \{\text{True}, \text{False}\}$$

che associa ad ogni sottoinsieme di attività $A \subseteq \{1, \dots, n\}$, *True* se A contiene attività mutualmente compatibili, *False* altrimenti. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$\text{COMP}(A) = \begin{cases} \text{True} & \text{se } \forall i, j \in A \text{ con } i \neq j, [s_i, e_i) \cap [s_j, e_j) = \emptyset \\ \text{False} & \text{altrimenti} \end{cases}$$

Infine, definiamo la funzione

$$V : \mathcal{P}(\{1, \dots, n\}) \rightarrow \mathbb{R}_+$$

che associa ad ogni sottoinsieme di attività $A \subseteq \{1, \dots, n\}$, il suo valore complessivo. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$V(A) = \begin{cases} \sum_{i \in A} v_i & \text{se } A \neq \emptyset \\ 0 & \text{se } A = \emptyset \end{cases}$$

11.2 Definizione del problema

Il problema PB è definito come segue:

dato un insieme $\{1, \dots, n\}$ di attività, trovare un suo sottoinsieme di attività mutualmente compatibili di valore massimo

Il problema può essere formalizzato come segue:

Istanza: $\{1, \dots, n\}$ con $([s_i, e_i], v_i), \forall i \in \{1, \dots, n\}$

Soluzione: $S \subseteq \{1, \dots, n\}$ tale che

$$\text{COMP}(S) = \text{True} \wedge \text{V}(S) = \max_{A \subseteq \{1, \dots, n\}: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

Si noti che, diversamente da alcuni dei problemi precedenti, risolveremo questo problema senza definire un problema ridotto: durante la risoluzione manterremo due variabili, una rappresentante il valore di un sottoinsieme di attività e una rappresentante il sottoinsieme di attività vero e proprio. Come vedremo, la prima ci servirà per determinare il valore della seconda.

11.3 Sottoproblemi e variabili associate

Il problema PB contiene diversi sottoproblemi ognuno dei quali non ha come input l'insieme $\{1, \dots, n\}$ con $([s_i, e_i], v_i), \forall i \in \{1, \dots, n\}$ ma un suo sottoinsieme:

1. dato $X_n = \{1, \dots, n\}$, si vuole trovare $S_n \subseteq X_n$ tale che

$$\text{COMP}(S_n) = \text{True} \wedge \text{V}(S_n) = \max_{A \subseteq X_n: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

2. dato $X_{n-1} = \{1, \dots, n-1\}$, si vuole trovare $S_{n-1} \subseteq X_{n-1}$ tale che

$$\text{COMP}(S_{n-1}) = \text{True} \wedge \text{V}(S_{n-1}) = \max_{A \subseteq X_{n-1}: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

3. ...

4. dato $X_2 = \{1, 2\}$, si vuole trovare $S_2 \subseteq X_2$ tale che

$$\text{COMP}(S_2) = \text{True} \wedge \text{V}(S_2) = \max_{A \subseteq X_2: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

5. dato $X_1 = \{1\}$, si vuole trovare $S_1 \subseteq X_1$ tale che

$$\text{COMP}(S_1) = \text{True} \wedge \text{V}(S_1) = \max_{A \subseteq X_1: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

6. dato $X_0 = \emptyset$, si vuole trovare $S_0 \subseteq X_0$ tale che

$$\text{COMP}(S_0) = \text{True} \wedge \text{V}(S_0) = \max_{A \subseteq X_0: \text{COMP}(A) = \text{True}} \{\text{V}(A)\}$$

Quindi, il generico sottoproblema del problema PB è individuato da $i \in \{1, \dots, n\}$.

Il **sottoproblema di dimensione i** è definito come segue:

dato un insieme $\{1, \dots, i\}$ di attività, trovare un suo sottoinsieme di attività mutualmente compatibili di valore massimo

ossia:

Istanza: $\{1, \dots, i\}$ con $([s_j, e_j], v_j), \forall j \in \{1, \dots, i\}$

Soluzione: $S_i \subseteq \{1, \dots, i\}$ tale che

$$\text{COMP}(S_i) = \text{True} \wedge V(S_i) = \max_{A \subseteq \{1, \dots, i\}: \text{COMP}(A) = \text{True}} \{V(A)\}$$

Dato che $0 \leq i \leq n$ si ottengono $(n+1)$ sottoproblemi, ad ognuno dei quali è associata una **coppia di variabili**.

Considerato il sottoproblema di dimensione i , la coppia di variabili ad esso associata è (OPT_i, S_i) ed è così definita:

$\text{OPT}_i = V(S_i)$, ossia il valore di un sottoinsieme di $\{1, \dots, i\}$ che contiene attività mutualmente compatibili di peso massimo
 S_i , un sottoinsieme di $\{1, \dots, i\}$ che contiene attività mutualmente compatibili di peso massimo, pari a OPT_i

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione i , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione 4, il valore della variabile OPT_4 si otterrà utilizzando le variabili OPT_0 , OPT_1 , OPT_2 e OPT_3 mentre l'insieme S_4 si otterrà utilizzando sia i vari OPT_0 , OPT_1 , OPT_2 e OPT_3 che i sottoinsiemi S_0 , S_1 , S_2 e S_3 . Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione i , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{(\text{OPT}_0, S_0), \dots, (\text{OPT}_{i-1}, S_{i-1})\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione (OPT_i, S_i) andando a combinare le soluzioni dei sottoproblemi di dimensione minore a i ed, eventualmente, utilizzando l'attività i estratta dall'input $\{1, \dots, i\}$?

11.4 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

11.4.1 Caso base: $i = 0$

Il caso base si ha per un qualunque sottoproblema di dimensione $i = 0$, ossia quando non ci sono attività da considerare. In questo caso, è facile ottenere la soluzione del sottoproblema: $\text{OPT}_i = 0$ e, di conseguenza, $S_i = \emptyset$. Dunque, il caso base è scrivibile come:

$$\text{OPT}_i = 0 \text{ e } S_i = \emptyset \quad \text{se } i = 0$$

11.4.2 Passo ricorsivo: $i > 0$

Per definire il passo ricorsivo, assumiamo che le attività $\{1, \dots, n\}$ siano ordinate rispetto ai tempi di fine: $e_1 \leq e_2 \leq \dots \leq e_i \leq \dots \leq e_n$. Inoltre, dato $i \in \{1, \dots, n\}$, definiamo

$$\psi(i) = \max\{j | j < i \text{ e } j \text{ è compatibile con } i\}$$

Assumiamo che $\max\{\emptyset\} = 0$.

Ora possiamo definire il passo ricorsivo. Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione i con $i > 0$, ossia quando si va a considerare un insieme che contiene almeno un'attività. I dati disponibili per calcolare la soluzione di un sottoproblema sono: l'input $\{1, \dots, i\}$ (in particolare l'attività i) e tutte le soluzioni dei sottoproblemi di dimensione minore. Per risolvere un sottoproblema è necessario domandarsi: “Come è fatto S_i rispetto a $S_{i-1}, S_{i-2}, \dots, S_1, S_0$?”. Per rispondere a questa domanda, bisogna considerare due casi:

- $i \notin S_i$: se l'attività i non appartiene alla soluzione, allora è sufficiente considerare l'insieme di attività $\{1, \dots, i-1\}$ e si può dunque considerare la soluzione del sottoproblema di dimensione subito minore, ossia

$$\text{OPT}_i = \text{OPT}_{i-1} \text{ e } S_i = S_{i-1}$$

- $i \in S_i$: se l'attività i appartiene alla soluzione, allora è necessario andare a determinare la soluzione del sottoproblema di dimensione minore ad i di peso massimo e il cui insieme di attività risulta essere compatibile con l'attività i . È dunque possibile considerare la soluzione del sottoproblema di dimensione $\psi(i)$ e aggiungere a questa l'attività i . Si ottiene dunque:

$$\text{OPT}_i = \text{OPT}_{\psi(i)} + v_i \text{ e } S_i = S_{\psi(i)} \cup \{i\}$$

Siccome non possiamo sapere a priori se l'attività i appartenga o meno alla soluzione, consideriamo entrambe le soluzioni e scegliamo quella di valore massimo. Formalmente, possiamo scrivere:

$$\text{OPT}_i = \max\{\text{OPT}_{i-1}, \text{OPT}_{\psi(i)} + v_i\}$$

e

$$S_i = \begin{cases} S_{i-1} & \text{se } \text{OPT}_{i-1} \geq \text{OPT}_{\psi(i)} + v_i \\ S_{\psi(i)} \cup \{i\} & \text{altrimenti} \end{cases}$$

Si noti come l'insieme S_i venga definito a partire dalle variabili S_j ma sulla base dei valori delle variabili OPT_j relative ai sottoproblemi di dimensione minore (ossia con $j < i$).

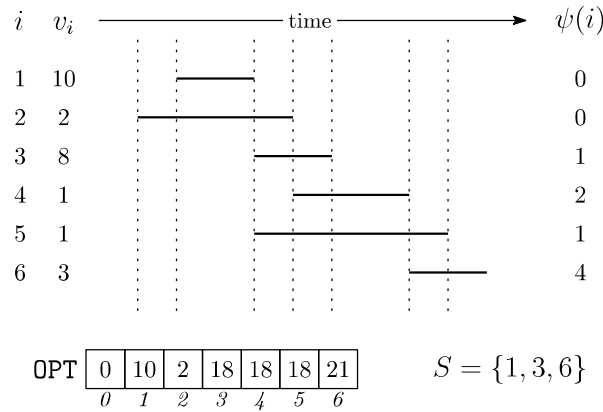


Figura 8: Weighted Interval Scheduling - Esempio

11.5 Soluzione del problema

Una volta calcolati i valori $\text{OPT}_0, \text{OPT}_1, \dots, \text{OPT}_n$, è possibile determinare la soluzione al problema PB andando a considerare (OPT_n, S_n) .

Esempio Un esempio è mostrato in Figura 8.

11.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 11.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione di ogni sottoproblema. L'algoritmo 21 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{WISRIC}(n)$. In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 21 Weighted interval scheduling (tecnica: ricorsione).

```

procedure WISRIC(i)
  if  $i = 0$  then
    return  $(0, \emptyset)$ 
  else
     $(V_1, S_1) \leftarrow \text{WISRIC}(i - 1)$ 
     $(V_2, S_2) \leftarrow \text{WISRIC}(\psi(i))$ 
     $V_2 \leftarrow V_2 + v_i$ 
    if  $V_1 \geq V_2$  then
      return  $(V_1, S_1)$ 
    else
      return  $(V_2, S_2 \cup \{i\})$ 

```

11.7 Implementazione bottom-up

L'algoritmo 22 presenta lo pseudocodice che calcola tutti i valori (OPT_0, S_0) , (OPT_1, S_1) , $\dots$, (OPT_n, S_n) basandosi sulle equazioni di ricorrenza definite nella Sezione 11.4. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(n)$ occupando, però, $\mathcal{O}(n + n^2)$ spazio in memoria (ossia un vettore che contiene i vari OPT_i e un vettore che contiene i vari insiemi S_i , ognuno dei quali contiene al massimo n elementi).

Algorithm 22 Weighted interval scheduling (tecnica: bottom-up).

```
procedure WIS( $n$ )
   $\text{OPT}[0] \leftarrow 0$ 
   $S \leftarrow \emptyset$ 
  for  $i \leftarrow 1$  to  $n$  do
     $V_1 \leftarrow \text{OPT}[i - 1]$ 
     $V_2 \leftarrow \text{OPT}[\psi(i)] + v_i$ 
     $\text{OPT}[i] \leftarrow \text{MAX}(V_1, V_2)$ 
    if  $V_1 \geq V_2$  then
       $S[i] \leftarrow S[i - 1]$ 
    else
       $S[i] \leftarrow S[\psi(i)] \cup \{i\}$ 
  return  $(\text{OPT}[n], S[n])$ 
```

12 Problema dello zaino

12.1 Definizioni preliminari

Sia $C > 0$ la capacità di uno zaino e sia $X = \{1, 2, \dots, n\}$ un insieme di n oggetti, con $n \in \mathbb{N}$. Ad ogni oggetto $i \in X$ sono associate le seguenti informazioni (v_i, w_i) , dove:

- v_i indica il valore dell'oggetto
- w_i indica il peso dell'oggetto

Introduciamo ora la funzione

$$V : \mathcal{P}(\{1, \dots, n\}) \rightarrow \mathbb{R}_+$$

che associa ad ogni sottoinsieme di oggetti $A \subseteq \{1, \dots, n\}$, il suo valore complessivo. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$V(A) = \begin{cases} \sum_{i \in A} v_i & \text{se } A \neq \emptyset \\ 0 & \text{se } A = \emptyset \end{cases}$$

Introduciamo inoltre la funzione

$$W : \mathcal{P}(\{1, \dots, n\}) \rightarrow \mathbb{R}_+$$

che associa ad ogni sottoinsieme di oggetti $A \subseteq \{1, \dots, n\}$, il suo ingombro (peso) complessivo. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$W(A) = \begin{cases} \sum_{i \in A} w_i & \text{se } A \neq \emptyset \\ 0 & \text{se } A = \emptyset \end{cases}$$

12.2 Definizione del problema

Il problema PB è definito come segue:

dato un insieme $X = \{1, \dots, n\}$ di oggetti, trovare un suo sottoinsieme di ingombro complessivo $\leq C$ e di massimo valore complessivo.

Il problema può essere ridefinito come segue:

Istanza: $X = \{1, \dots, n\}$ con (v_i, w_i) , $\forall i \in \{1, \dots, n\}$

Soluzione: $S \subseteq \{1, \dots, n\}$ tale che

$$W(S) \leq C \wedge V(S) = \max_{A \subseteq \{1, \dots, n\}: W(A) \leq C} \{V(A)\}$$

Come per l'esercizio *Weighted Interval Scheduling*, risolveremo questo problema senza definire un problema ridotto.

12.3 Sottoproblemi e variabili associate (Falsa partenza)

Il problema PB contiene diversi sottoproblemi ognuno dei quali non ha come input l'insieme $\{1, \dots, n\}$ con (v_i, w_i) , $\forall i \in \{1, \dots, n\}$ ma un suo sottoinsieme:

1. dato $X_n = \{1, \dots, n\}$, si vuole trovare $S_n \subseteq X_n$ tale che

$$W(S_n) \leq C \wedge V(S_n) = \max_{A \subseteq X_n: W(A) \leq C} \{V(A)\}$$

2. dato $X_{n-1} = \{1, \dots, n-1\}$, si vuole trovare $S_{n-1} \subseteq X_{n-1}$ tale che

$$W(S_{n-1}) \leq C \wedge V(S_{n-1}) = \max_{A \subseteq X_{n-1}: W(A) \leq C} \{V(A)\}$$

3. ...

4. dato $X_i = \{1, \dots, i\}$, si vuole trovare $S_i \subseteq X_i$ tale che

$$W(S_i) \leq C \wedge V(S_i) = \max_{A \subseteq X_i: W(A) \leq C} \{V(A)\}$$

5. ...

6. dato $X_0 = \emptyset$, si vuole trovare $S_0 \subseteq X_0$ tale che

$$W(S_0) \leq C \wedge V(S_0) = \max_{A \subseteq X_0: W(A) \leq C} \{V(A)\}$$

Quindi, il generico sottoproblema del problema PB è individuato da $i \in \{1, \dots, n\}$.

Il **sottoproblema di dimensione i** è definito come segue:

dato un insieme $\{1, \dots, i\}$ di oggetti, trovare un suo sottoinsieme di ingombro complessivo $\leq C$ e di massimo valore complessivo.

ossia:

Istanza: $X_i = \{1, \dots, i\}$ con (v_j, w_j) , $\forall j \in \{1, \dots, i\}$

Soluzione: $S_i \subseteq \{1, \dots, i\}$ tale che

$$W(S_i) \leq C \wedge V(S_i) = \max_{A \subseteq \{1, \dots, i\}: W(A) \leq C} \{V(A)\}$$

Dato che $0 \leq i \leq n$ si ottengono $(n+1)$ sottoproblemi, ad ognuno dei quali è associata una **coppia di variabili**.

Considerato il sottoproblema di dimensione i , la coppia di variabili ad esso associata è (OPT_i, S_i) ed è così definita:

$OPT_i = V(S_i)$, *ossia il valore di un sottoinsieme di $\{1, \dots, i\}$ di ingombro complessivo $\leq C$ e di massimo valore complessivo.*

S_i , *un sottoinsieme di $\{1, \dots, i\}$ di ingombro complessivo $\leq C$ e di massimo valore complessivo, pari a OPT_i .*

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione i , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione 4, il valore della variabile OPT_4 si otterrà utilizzando le variabili OPT_0 , OPT_1 , OPT_2 e OPT_3 mentre l'insieme S_4 si otterrà utilizzando sia i vari OPT_0 , OPT_1 , OPT_2 e OPT_3 che i sottoinsiemi S_0 , S_1 , S_2 e S_3 . Si noti, però, che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione i , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{(\text{OPT}_0, S_0), \dots, (\text{OPT}_{i-1}, S_{i-1})\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione (OPT_i, S_i) andando a combinare le soluzioni dei sottoproblemi di dimensione minore a i ed, eventualmente, utilizzando l'oggetto i estratto dall'input $\{1, \dots, i\}$?

Purtroppo, però, il problema così definito non è risolvibile: **i sottoproblemi così definiti non ci permettono di risolvere il problema**. Date solamente le variabili $\{(\text{OPT}_0, S_0), \dots, (\text{OPT}_{i-1}, S_{i-1})\}$ e il sottoinsieme $\{1, \dots, i\}$ (del quale, in realtà, ci interessa solo l'oggetto i), non c'è alcun modo per poter comprendere se l'elemento i possa essere aggiunto a qualche sottoinsieme di oggetti relativo ai sottoproblemi di dimensione minore a i : non sappiamo l'ingombro complessivo di questi sottoinsiemi e quindi non possiamo sapere se, aggiungendo l'oggetto i , si superi la capacità C dello zaino.

Risulta pertanto necessario ridefinire i sottoproblemi.

12.4 Sottoproblemi e variabili associati

È necessario suddividere il problema PR in $(n + 1) \cdot (C + 1)$ sottoproblemi. Il sottoproblema di dimensione (i, c) , con $0 \leq i \leq n$ e $0 \leq c \leq C$ è definito come segue:

dato un insieme $\{1, \dots, i\}$ di oggetti, trovare un suo sottoinsieme di ingombro complessivo $\leq c$ e di massimo valore complessivo.

In altre parole, si richiede che il sottoinsieme scelto abbia ingombro complessivo minore a c . I sottoproblemi possono essere riscritti come:

Istanza: $X = \{1, \dots, i\}$ con (v_j, w_j) , $\forall j \in \{1, \dots, i\}$ e $c \in \{0, \dots, C\}$

Soluzione: $S \subseteq \{1, \dots, i\}$ tale che

$$W(S_{i,c}) \leq c \wedge V(S_{i,c}) = \max_{A \subseteq \{1, \dots, i\}: W(A) \leq c} \{V(A)\}$$

Considerato il sottoproblema di dimensione (i, c) , la variabile ad esso associata è $(\text{OPT}_{i,c}, S_{i,c})$ ed è così definita:

$\text{OPT}_{i,c} = V(S_{i,c})$, ossia il valore di un sottoinsieme di $\{1, \dots, i\}$ di ingombro complessivo $\leq c$ e di massimo valore complessivo.

$S_{i,c}$, un sottoinsieme di $\{1, \dots, i\}$ di ingombro complessivo $\leq C$ e di massimo valore complessivo, pari a $\text{OPT}_{i,c}$.

Si noti quindi che per un qualunque sottoproblema di dimensione (i, c) , solo imponendo che la soluzione abbia ingombro complessivo $\leq c$, è possibile stabilire se un altro elemento j possa essere aggiunto alla soluzione senza andare a superare la capacità complessiva dello zaino.

12.5 Equazioni di ricorrenza

È ora possibile definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

12.5.1 Caso base: $i = 0 \vee c = 0$

Il caso base si ha per un qualunque sottoproblema di dimensione (i, c) con $i = 0 \vee c = 0$. In questo caso, è facile ottenere la soluzione del sottoproblema. Infatti, se $i = 0$, non abbiamo oggetti mentre se $c = 0$ si richiede che l'insieme di oggetti abbia ingombro complessivo pari a 0. In entrambi questi casi, $\text{OPT}_{i,c} = 0$ e, di conseguenza, $S_{i,c} = \emptyset$. Dunque, il caso base è scrivibile come:

$$\text{OPT}_{i,c} = 0 \text{ e } S_{i,c} = \emptyset \quad \text{se } i = 0 \vee c = 0$$

12.5.2 Passo ricorsivo: $i > 0 \wedge c > 0$

Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, c) con $i > 0 \wedge c > 0$, ossia quando si va a considerare un insieme che contiene almeno un oggetto e si richiede un ingombro complessivo minore di un certo valore diverso da 0. I dati disponibili per calcolare la soluzione di un sottoproblema sono: l'input $\{1, \dots, i\}$ (in particolare l'oggetto i) e tutte le soluzioni dei sottoproblemi di dimensione minore (ossia $\text{OPT}_{0,0}, \dots, \text{OPT}_{i-1,c-1}, \text{OPT}_{i-1,c}, \text{OPT}_{i,c-1}$). È necessario considerare due casi:

- $w_i > c$: se l'oggetto ha ingombro maggiore della capacità, allora l'oggetto non può appartenere alla soluzione e quindi si va a considerare la soluzione del sottoproblema di dimensione minore $i-1, c$ (non si considera l'oggetto i ma si considera la stessa capacità). In questo caso:

$$\text{OPT}_{i,c} = \text{OPT}_{i-1,c} \text{ e } S_{i,c} = S_{i-1,c} \quad \text{se } w_i > c$$

- $w_i \leq c$: se l'oggetto ha ingombro minore o uguale della capacità, allora l'oggetto potrebbe appartenere alla soluzione. Ci si può quindi trovare in due sottocasi:

- $i \notin S_i$: se l'oggetto i non appartiene alla soluzione, allora è sufficiente considerare l'insieme di oggetti $\{1, \dots, i-1\}$ e si può dunque considerare la soluzione del sottoproblema di dimensione $i-1, c$, ossia

$$\text{OPT}_{i,c} = \text{OPT}_{i-1,c} \text{ e } S_{i,c} = S_{i-1,c}$$

- $i \in S_i$: se l'oggetto i appartiene alla soluzione, allora è necessario andare a determinare la soluzione del sottoproblema di dimensione minore a (i, c) di valore massimo e il cui insieme di oggetti risulta essere compatibile con l'oggetto i : ossia è possibile aggiungere l'oggetto i senza superare la capacità c . È dunque possibile considerare la soluzione del sottoproblema di dimensione $(i-1, c-w_i)$ e aggiungere a questo l'oggetto i (si noti come alla capacità sia stato tolto il peso dell'oggetto che si vuole aggiungere). Si ottiene dunque:

$$\text{OPT}_{i,c} = \text{OPT}_{i-1,c-w_i} + v_i \text{ e } S_{i,c} = S_{i-1,c-w_i} \cup \{i\}$$

Siccome non possiamo sapere a priori se l'oggetto i appartenga o meno alla soluzione, consideriamo entrambe le soluzioni e scegliamo quella di valore massimo. Dunque, se $w_i \leq c$, possiamo scrivere:

$$\text{OPT}_{i,c} = \max\{\text{OPT}_{i-1,c}, \text{OPT}_{i-1,c-w_i} + v_i\}$$

e

$$S_{i,c} = \begin{cases} S_{i-1,c} & \text{se } \text{OPT}_{i-1,c} \geq \text{OPT}_{i-1,c-w_i} + v_i \\ S_{i-1,c-w_i} \cup \{i\} & \text{altrimenti} \end{cases}$$

Dunque, il passo ricorsivo può essere riscritto come:

$$\text{OPT}_{i,c} = \begin{cases} \text{OPT}_{i-1,c} & \text{se } w_i > c \\ \max\{\text{OPT}_{i-1,c}, \text{OPT}_{i-1,c-w_i} + v_i\} & \text{altrimenti} \end{cases}$$

e

$$S_{i,c} = \begin{cases} S_{i-1,c} & \text{se } w_i > c \\ \begin{cases} S_{i-1,c} & \text{se } \text{OPT}_{i-1,c} \geq \text{OPT}_{i-1,c-w_i} + v_i \\ S_{i-1,c-w_i} \cup \{i\} & \text{altrimenti} \end{cases} & \text{altrimenti} \end{cases}$$

12.6 Soluzione del problema

Una volta calcolati i valori $\text{OPT}_{0,0}, \dots, \text{OPT}_{n,C}$, è possibile determinare la soluzione al problema PB andando a considerare $(\text{OPT}_{n,C}, S_{n,C})$.

Esempio Si consideri l'insieme di oggetti presentati in Tabella 1. Le soluzioni $\text{OPT}_{i,c}$ di tutti i sottoproblemi sono presentate in Tabella 2.

12.7 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 12.5, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione di ogni sottoproblema. L'algoritmo 23 ne presenta lo pseudocodice: la prima chiamata deve essere

i	v_i	w_i
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Tabella 1: Problema dello zaino - Input.

$i \backslash c$	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	18	22	24	28	28	29	40
5	0	1	6	7	7	18	22	28	29	34	35	40

Tabella 2: Problema dello zaino: matrice delle soluzioni $\text{OPT}_{i,c}$ ottenuta considerando l'input presentato in Tabella 1 e una capacità dello zaino pari a $C = 11$.

$\text{KPRIC}(n, C)$. In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 23 Problema dello zaino (tecnica: ricorsione).

```

procedure KPRIC( $i, c$ )
  if  $i = 0 \vee c = 0$  then
    return  $(0, \emptyset)$ 
  else
    if  $w_i > c$  then
      return KPRIC( $i - 1, c$ )
    else
       $(V_1, S_1) \leftarrow \text{KPRIC}(i - 1, c)$ 
       $(V_2, S_2) \leftarrow \text{KPRIC}(i - 1, c - w_i)$ 
       $V_2 \leftarrow V_2 + v_i$ 
      if  $V_1 \geq V_2$  then
        return  $(V_1, S_1)$ 
      else
        return  $(V_2, S_2 \cup \{i\})$ 

```

12.8 Implementazione bottom-up

L'algoritmo 24 presenta lo pseudocodice che calcola tutti i valori (OPT_0, S_0) , (OPT_1, S_1) , $\dots$, (OPT_n, S_n) basandosi sulle equazioni di ricorrenza definite nella Sezione 12.5. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(n \cdot C)$ occupando, però, $\Theta(n \cdot C + n^2 \cdot C)$ spazio in memoria (ossia una matrice che contiene i vari $\text{OPT}_{i,c}$ e una matrice che contiene i vari insiemi $S_{i,c}$, ognuno dei quali contiene al massimo n elementi).

Osservazione: questo algoritmo non è polinomiale nell'input ma pseudopolinomiale.

Algorithm 24 Problema dello zaino (tecnica: bottom-up).

```
procedure KP( $n, C$ )
  for  $i \leftarrow 0$  to  $n$  do
     $\text{OPT}[i, 0] \leftarrow 0$ 
     $S[i, 0] \leftarrow \emptyset$ 
  for  $c \leftarrow 0$  to  $C$  do
     $\text{OPT}[0, c] \leftarrow 0$ 
     $S[0, c] \leftarrow \emptyset$ 
  for  $i \leftarrow 1$  to  $n$  do
    for  $c \leftarrow 1$  to  $C$  do
      if  $w_i > c$  then
         $\text{OPT}[i, c] \leftarrow \text{OPT}[i - 1, c]$ 
         $S[i, c] \leftarrow S[i - 1, c]$ 
      else
         $V_1 \leftarrow \text{OPT}[i - 1, c]$ 
         $V_2 \leftarrow \text{OPT}[i - 1, c - w_i] + v_i$ 
         $\text{OPT}[i, c] \leftarrow \text{MAX}(V_1, V_2)$ 
        if  $V_1 \geq V_2$  then
           $S[i, c] \leftarrow S[i - 1, c]$ 
        else
           $S[i, c] \leftarrow S[i - 1, c - w_i] \cup \{i\}$ 
  return  $(\text{OPT}[n, C], S[n, C])$ 
```

13 Constrained Weighted Interval Scheduling

Questo esercizio può essere visto come la combinazione del problema *Weighted Interval Scheduling* con il problema dello zaino.

13.1 Definizioni preliminari

Sia $A = \{1, 2, \dots, n\}$ un insieme di n attività, con $n \in \mathbb{N}$. Ad ogni attività $i \in A$ sono associate le seguenti informazioni $([s_i, e_i], v_i, w_i)$, dove:

- s_i indica il tempo di inizio dell'attività
- e_i , con $e_i \geq s_i$, indica il tempo di fine dell'attività (si noti che e_i non appartiene all'intervallo che specifica la durata dell'attività)
- v_i indica il valore dell'attività
- w_i indica il peso (costo) dell'attività

Due attività $i, j \in A$ sono dette *compatibili* se non si sovrappongono:

$$[s_i, e_i) \cap [s_j, e_j) = \emptyset$$

Un insieme A di attività *contiene attività mutualmente compatibili* se e solo se per tutte le coppie (i, j) con $i \neq j$, l'attività i è compatibile con l'attività j :

$$\forall i, j \in A \text{ con } i \neq j, [s_i, e_i) \cap [s_j, e_j) = \emptyset$$

Definiamo ora la funzione

$$\text{COMP} : \mathcal{P}(\{1, \dots, n\}) \rightarrow \{\text{True}, \text{False}\}$$

che associa ad ogni sottoinsieme di attività $A \subseteq \{1, \dots, n\}$, *True* se A contiene attività mutualmente compatibili, *False* altrimenti. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$\text{COMP}(A) = \begin{cases} \text{True} & \text{se } \forall i, j \in A \text{ con } i \neq j, [s_i, e_i) \cap [s_j, e_j) = \emptyset \\ \text{False} & \text{altrimenti} \end{cases}$$

Poi, definiamo la funzione

$$\mathbf{V} : \mathcal{P}(\{1, \dots, n\}) \rightarrow \mathbb{R}_+$$

che associa ad ogni sottoinsieme di attività $A \subseteq \{1, \dots, n\}$, il suo valore complessivo. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$\mathbf{V}(A) = \begin{cases} \sum_{i \in A} v_i & \text{se } A \neq \emptyset \\ 0 & \text{se } A = \emptyset \end{cases}$$

Infine, definiamo la funzione

$$\mathbf{W} : \mathcal{P}(\{1, \dots, n\}) \rightarrow \mathbb{R}_+$$

che associa ad ogni sottoinsieme di attività $A \subseteq \{1, \dots, n\}$, il suo peso complessivo. Formalmente, $\forall A \subseteq \{1, \dots, n\}$:

$$\mathbf{W}(A) = \begin{cases} \sum_{i \in A} w_i & \text{se } A \neq \emptyset \\ 0 & \text{se } A = \emptyset \end{cases}$$

13.2 Definizione del problema

Il problema PB è definito come segue:

dato un insieme $\{1, \dots, n\}$ di attività, trovare un suo sottoinsieme di attività mutualmente compatibili di valore massimo e di peso complessivo $\leq C$.

Il problema può essere formalizzato come segue:

Istanza: $\{1, \dots, n\}$ con $([s_i, e_i), v_i, w_i), \forall i \in \{1, \dots, n\}$ e $C \in \mathbb{R}_+$

Soluzione: $S \subseteq \{1, \dots, n\}$ tale che

$$\text{COMP}(S) = \text{True} \wedge \text{W}(S) \leq C \wedge \text{V}(S) = \max_{A \subseteq \{1, \dots, n\}: \text{COMP}(A) = \text{True} \wedge \text{W}(A) \leq C} \{\text{V}(A)\}$$

Come per l'esercizio *Weighted Interval Scheduling*, risolveremo questo problema senza definire un problema ridotto. Inoltre, come è successo per il problema dello zaino, si potrebbe partire nel modo sbagliato: per brevità, per questo esercizio, non viene riportata la falsa partenza.

13.3 Sottoproblemi e variabili associate

Come per il problema dello zaino, questo problema può essere suddiviso in $(n+1) \cdot (C+1)$ sottoproblemi. Ogni sottoproblema di dimensione (i, c) , con $0 \leq i \leq n$ e $0 \leq c \leq C$ è definito come segue:

dato un insieme $\{1, \dots, i\}$ di attività, trovare un suo sottoinsieme di attività mutualmente compatibili di valore massimo e di peso complessivo $\leq c$.

ossia:

Istanza: $\{1, \dots, i\}$ con $([s_j, e_j), v_j, w_j), \forall j \in \{1, \dots, i\}$ e $c \in \{0, \dots, C\}$

Soluzione: $S \subseteq \{1, \dots, i\}$ tale che

$$\text{COMP}(S) = \text{True} \wedge \text{W}(S) \leq c \wedge \text{V}(S) = \max_{A \subseteq \{1, \dots, i\}: \text{COMP}(A) = \text{True} \wedge \text{W}(A) \leq c} \{\text{V}(A)\}$$

Considerato il sottoproblema di dimensione (i, c) , la variabile ad esso associata è $(\text{OPT}_{i,c}, S_{i,c})$ ed è così definita:

$\text{OPT}_{i,c} = \text{V}(S_{i,c})$, ossia il valore di un sottoinsieme di $\{1, \dots, i\}$ di attività mutualmente compatibili di peso complessivo $\leq c$ e di massimo valore complessivo.

$S_{i,c}$, un sottoinsieme di $\{1, \dots, i\}$ di attività mutualmente compatibili di peso complessivo $\leq c$ e di massimo valore complessivo.

Per determinare la soluzione di un qualsiasi sottoproblema di dimensione i, c , oltre all'input del problema, si utilizzeranno le soluzioni dei sottoproblemi di dimensione minore. Per esempio, dato il sottoproblema di dimensione $(2, 1)$, il valore della variabile $\text{OPT}_{2,1}$ si otterrà utilizzando le variabili $\text{OPT}_{0,0}$, $\text{OPT}_{0,1}$, $\text{OPT}_{1,0}$ e $\text{OPT}_{1,1}$ mentre l'insieme $S_{2,1}$ si otterrà utilizzando sia i vari $\text{OPT}_{0,0}$, $\text{OPT}_{0,1}$, $\text{OPT}_{1,0}$ e $\text{OPT}_{1,1}$ che i sottoinsiemi $S_{0,0}$, $S_{0,1}$, $S_{1,0}$ e $S_{1,1}$. Si noti, però,

che ognuna di queste variabili è da considerare come una **black-box**: si può utilizzare ma non è possibile conoscerne il contenuto.

Considerato un qualunque sottoproblema di dimensione i , assumendo di aver già risolto tutti i sottoproblemi di dimensione minore (ossia sapendo di poter utilizzare le variabili $\{(\text{OPT}_0, S_0), \dots, (\text{OPT}_{i-1}, S_{i-1})\}$ senza però sapere effettivamente quanto valgono), come è possibile ottenere la soluzione (OPT_i, S_i) andando a combinare le soluzioni dei sottoproblemi di dimensione minore a i ed, eventualmente, utilizzando l'attività i estratta dall'input $\{1, \dots, i\}$?

13.4 Equazioni di ricorrenza

Per rispondere a questa domanda, si devono andare a definire le **equazioni di ricorrenza**.

Le equazioni di ricorrenza si scrivono senza sapere il valore delle soluzioni dei sottoproblemi ma sapendo solamente che tali soluzioni esistono e si possono utilizzare

Un'equazione di ricorrenza è composta da:

- un **caso base** che definisce i casi più semplici che possono essere subito risolti senza ricorrere alle soluzioni dei sottoproblemi più piccoli
- un **passo ricorsivo** che definisce come risolvere i casi più complessi a partire dalle soluzioni dei sottoproblemi più piccoli (che si assume essere stati già risolti)

13.4.1 Caso base: $i = 0 \vee c = 0$

Il caso base si ha per un qualunque sottoproblema di dimensione (i, c) con $i = 0 \vee c = 0$. In questo caso, è facile ottenere la soluzione del sottoproblema. Infatti, se $i = 0$, non abbiamo attività mentre se $c = 0$ si richiede che l'insieme di attività abbia peso complessivo pari a 0. In entrambi questi casi, $\text{OPT}_{i,c} = 0$ e, di conseguenza, $S_{i,c} = \emptyset$. Dunque, il caso base è scrivibile come:

$$\text{OPT}_{i,c} = 0 \text{ e } S_{i,c} = \emptyset \quad \text{se } i = 0 \vee c = 0$$

13.4.2 Passo ricorsivo: $i > 0 \wedge c > 0$

Per definire il passo ricorsivo, assumiamo che le attività $\{1, \dots, n\}$ siano ordinate rispetto ai tempi di fine: $e_1 \leq e_2 \leq \dots \leq e_i \leq \dots \leq e_n$. Inoltre, dato $i \in \{1, \dots, n\}$, definiamo

$$\psi(i) = \max\{j | j < i \text{ e } j \text{ è compatibile con } i\}$$

Assumiamo che $\max\{\emptyset\} = 0$.

Ora possiamo definire il passo ricorsivo. Il passo ricorsivo si ha per un qualunque sottoproblema di dimensione (i, c) con $i > 0 \wedge c > 0$, ossia quando si va a considerare un insieme che contiene almeno un'attività e si richiede che il peso complessivo sia minore o uguale di un certo valore diverso da 0. I dati disponibili per calcolare la soluzione di un sottoproblema sono: l'input $\{1, \dots, i\}$ (in

particolare l'attività i) e tutte le soluzioni dei sottoproblemi di dimensione minore (ossia $\text{OPT}_{0,0}, \dots, \text{OPT}_{i-1,c-1}, \text{OPT}_{i-1,c}, \text{OPT}_{i,c-1}$). È necessario considerare due casi:

- $w_i > c$: se l'attività ha peso maggiore della capacità, allora l'attività non può appartenere alla soluzione e quindi si va a considerare la soluzione del sottoproblema di dimensione minore $i-1, c$ (non si considera l'attività i ma si considera la stessa capacità). In questo caso:

$$\text{OPT}_{i,c} = \text{OPT}_{i-1,c} \text{ e } S_{i,c} = S_{i-1,c} \quad \text{se } w_i > c$$

- $w_i \leq c$: se l'attività ha peso minore o uguale della capacità, allora potrebbe appartenere alla soluzione. Ci si può quindi trovare in due sottocasi:

- $i \notin S_i$: se l'attività i non appartiene alla soluzione, allora è sufficiente considerare l'insieme di attività $\{1, \dots, i-1\}$ e si può dunque considerare la soluzione del sottoproblema di dimensione $i-1, c$, ossia

$$\text{OPT}_{i,c} = \text{OPT}_{i-1,c} \text{ e } S_{i,c} = S_{i-1,c}$$

- $i \in S_i$: se l'attività i appartiene alla soluzione, allora è necessario andare a determinare la soluzione del sottoproblema di dimensione minore a (i, c) di valore massimo e il cui insieme di attività risulta essere compatibile con l'attività i : aggiungendo l'attività i , la soluzione rimane un insieme di attività mutualmente esclusive e non si supera la capacità c . È dunque possibile considerare la soluzione del sottoproblema di dimensione $(\psi(i), c - w_i)$ e aggiungere a questa l'attività i . Si ottiene dunque:

$$\text{OPT}_{i,c} = \text{OPT}_{\psi(i), c-w_i} + v_i \text{ e } S_{i,c} = S_{\psi(i), c-w_i} \cup \{i\}$$

Siccome non possiamo sapere a priori se l'attività i appartenga o meno alla soluzione, consideriamo entrambe le soluzioni e scegliamo quella di valore massimo. Dunque, se $w_i \leq c$, possiamo scrivere:

$$\text{OPT}_{i,c} = \max\{\text{OPT}_{i-1,c}, \text{OPT}_{\psi(i), c-w_i} + v_i\}$$

e

$$S_{i,c} = \begin{cases} S_{i-1,c} & \text{se } \text{OPT}_{i-1,c} \geq \text{OPT}_{\psi(i), c-w_i} + v_i \\ S_{\psi(i), c-w_i} \cup \{i\} & \text{altrimenti} \end{cases}$$

Dunque, il passo ricorsivo può essere riscritto come:

$$\text{OPT}_{i,c} = \begin{cases} \text{OPT}_{i-1,c} & \text{se } w_i > c \\ \max\{\text{OPT}_{i-1,c}, \text{OPT}_{\psi(i), c-w_i} + v_i\} & \text{altrimenti} \end{cases}$$

e

$$S_{i,c} = \begin{cases} S_{i-1,c} & \text{se } w_i > c \\ \begin{cases} S_{i-1,c} & \text{se } \text{OPT}_{i-1,c} \geq \text{OPT}_{\psi(i), c-w_i} + v_i \\ S_{\psi(i), c-w_i} \cup \{i\} & \text{altrimenti} \end{cases} & \text{altrimenti} \end{cases}$$

13.5 Soluzione del problema

Una volta calcolati i valori $\text{OPT}_{0,0}, \dots, \text{OPT}_{n,C}$, è possibile determinare la soluzione al problema PB andando a considerare $(\text{OPT}_{n,C}, S_{n,C})$.

13.6 Algoritmo ricorsivo

Utilizzando le equazioni di ricorrenza presentate nella Sezione 13.4, è immediato ricavare l'algoritmo ricorsivo per determinare la soluzione di ogni sottoproblema. L'algoritmo 25 ne presenta lo pseudocodice: la prima chiamata deve essere $\text{CWISRIC}(n, C)$. In ogni caso, questo algoritmo presenta una complessità in tempo esponenziale, dato che ogni sottoproblema viene risolto più volte. Per questa ragione, si utilizza una tecnica bottom-up per risolvere il problema: questa tecnica permette di calcolare la soluzione di ogni sottoproblema solamente una volta. In questo modo, si ottiene un algoritmo con complessità in spazio maggiore (in quanto si devono memorizzare le varie soluzioni) ma complessità in tempo minore rispetto l'algoritmo ricorsivo.

Algorithm 25 Constrained weighted interval scheduling (tecnica: ricorsione).

```
procedure CWISRIC(i,c)
  if  $i = 0 \vee c = 0$  then
    return  $(0, \emptyset)$ 
  else
    if  $w_i > c$  then
      return CWISRIC( $i - 1, c$ )
    else
       $(V_1, S_1) \leftarrow \text{CWISRIC}(i - 1, c)$ 
       $(V_2, S_2) \leftarrow \text{CWISRIC}(\psi(i), c - w_i)$ 
       $V_2 \leftarrow V_2 + v_i$ 
      if  $V_1 \geq V_2$  then
        return  $(V_1, S_1)$ 
      else
        return  $(V_2, S_2 \cup \{i\})$ 
```

13.7 Implementazione bottom-up

L'algoritmo 26 presenta lo pseudocodice che calcola tutti i valori (OPT_0, S_0) , (OPT_1, S_1) , $\dots$, (OPT_n, S_n) basandosi sulle equazioni di ricorrenza definite nella Sezione 13.4. Anziché utilizzare la ricorsione pura, tutti i valori sono calcolati con tecnica **bottom-up** in modo tale da risolvere ogni sottoproblema una volta sola. Questo permette di risolvere il problema in $\mathcal{O}(n \cdot C)$ occupando, però, $\Theta(n \cdot C + n^2 \cdot C)$ spazio in memoria (ossia una matrice che contiene i vari $\text{OPT}_{i,c}$ e una matrice che contiene i vari insiemi $S_{i,c}$, ognuno dei quali contiene al massimo n elementi).

Algorithm 26 Constrained weighted interval scheduling (tecnica: bottom-up).

```

procedure CWIS( $n, C$ )
  for  $i \leftarrow 0$  to  $n$  do
     $\text{OPT}[i, 0] \leftarrow 0$ 
     $S[i, 0] \leftarrow \emptyset$ 
  for  $c \leftarrow 0$  to  $C$  do
     $\text{OPT}[0, c] \leftarrow 0$ 
     $S[0, c] \leftarrow \emptyset$ 
  for  $i \leftarrow 1$  to  $n$  do
    for  $c \leftarrow 1$  to  $C$  do
      if  $w_i > c$  then
         $\text{OPT}[i, c] \leftarrow \text{OPT}[i - 1, c]$ 
         $S[i, c] \leftarrow S[i - 1, c]$ 
      else
         $V_1 \leftarrow \text{OPT}[i - 1, c]$ 
         $V_2 \leftarrow \text{OPT}[\psi(i), c - w_i] + v_i$ 
         $\text{OPT}[i, c] \leftarrow \text{MAX}(V_1, V_2)$ 
        if  $V_1 \geq V_2$  then
           $S[i, c] \leftarrow S[i - 1, c]$ 
        else
           $S[i, c] \leftarrow S[\psi(i), c - w_i] \cup \{i\}$ 
  return ( $\text{OPT}[n, C], S[n, C]$ )

```
